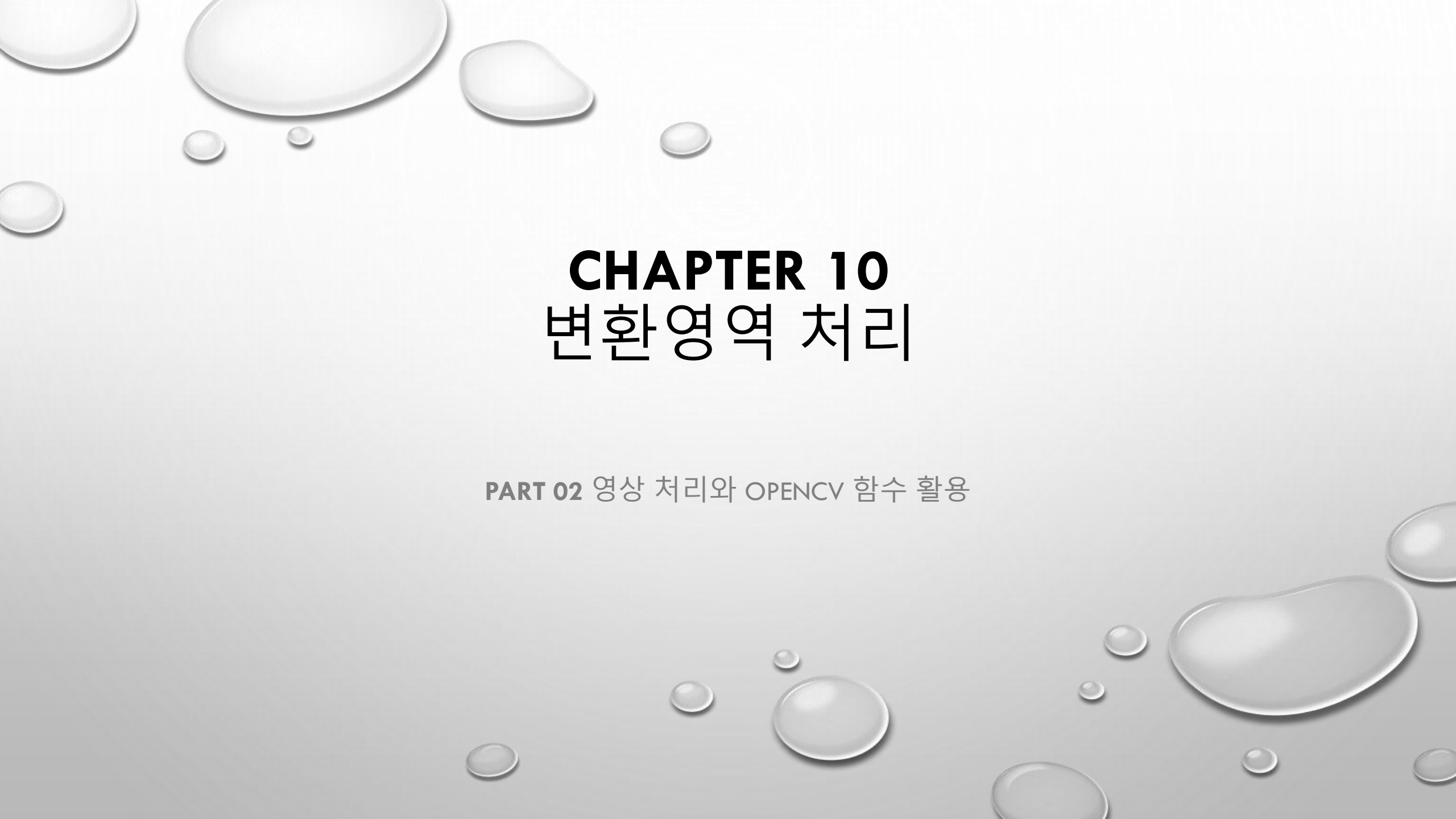




CHAPTER 10

변환영역 처리

PART 02 영상 처리와 OPENCV 함수 활용



CONTENTS

- ◆ 10.1 허프 변환
- ◆ 10.2 코너 검출
- ◆ 10.3 K-최근접 이웃 분류기
- ◆ 10.4 영상 워핑과 영상 모핑

10.1 허프 변환

- ◆10.1.1 허프 변환의 좌표계
- ◆10.1.2 허프 변환의 전체 과정
- ◆10.1.3 허프 누적 행렬 구성
- ◆10.1.4 허프 누적 행렬의 지역 최대값 선정
- ◆10.1.5 임계값 이상인 누적값(직선) 선별
- ◆10.1.6 직선을 누적값 기준으로 내림차순 정렬
- ◆10.1.7 최종 완성 프로그램
- ◆10.1.8 멀티 하네스의 전처리

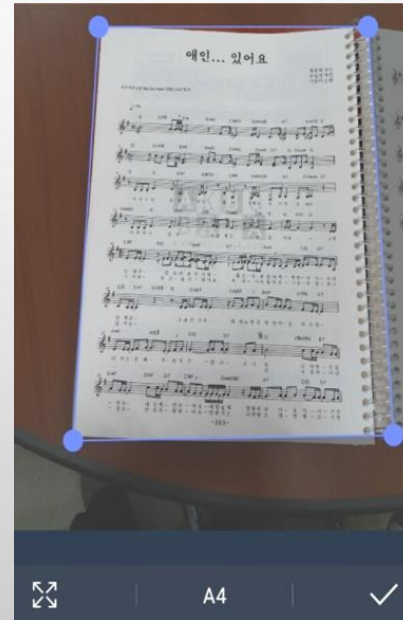
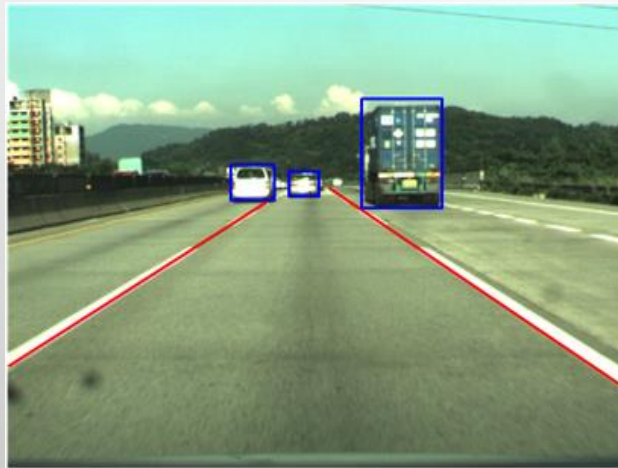
10.1 허프 변환

◆ 직선 검출

- ❖ 영상 내에서 공간 구조를 분석하는데 유용한 도구
- ❖ 영상 처리와 컴퓨터 비전분야에서 많은 연구 진행

◆ 다양한 응용에 사용

- ❖ 차선 및 장애물 자동인식 시스템 - 차선 검출
- ❖ 스캐너의 기능을 대신해 주는 앱 - 네 개 모서리 검출



10.1.1 허프 변환의 좌표계

◆ 허프 변환

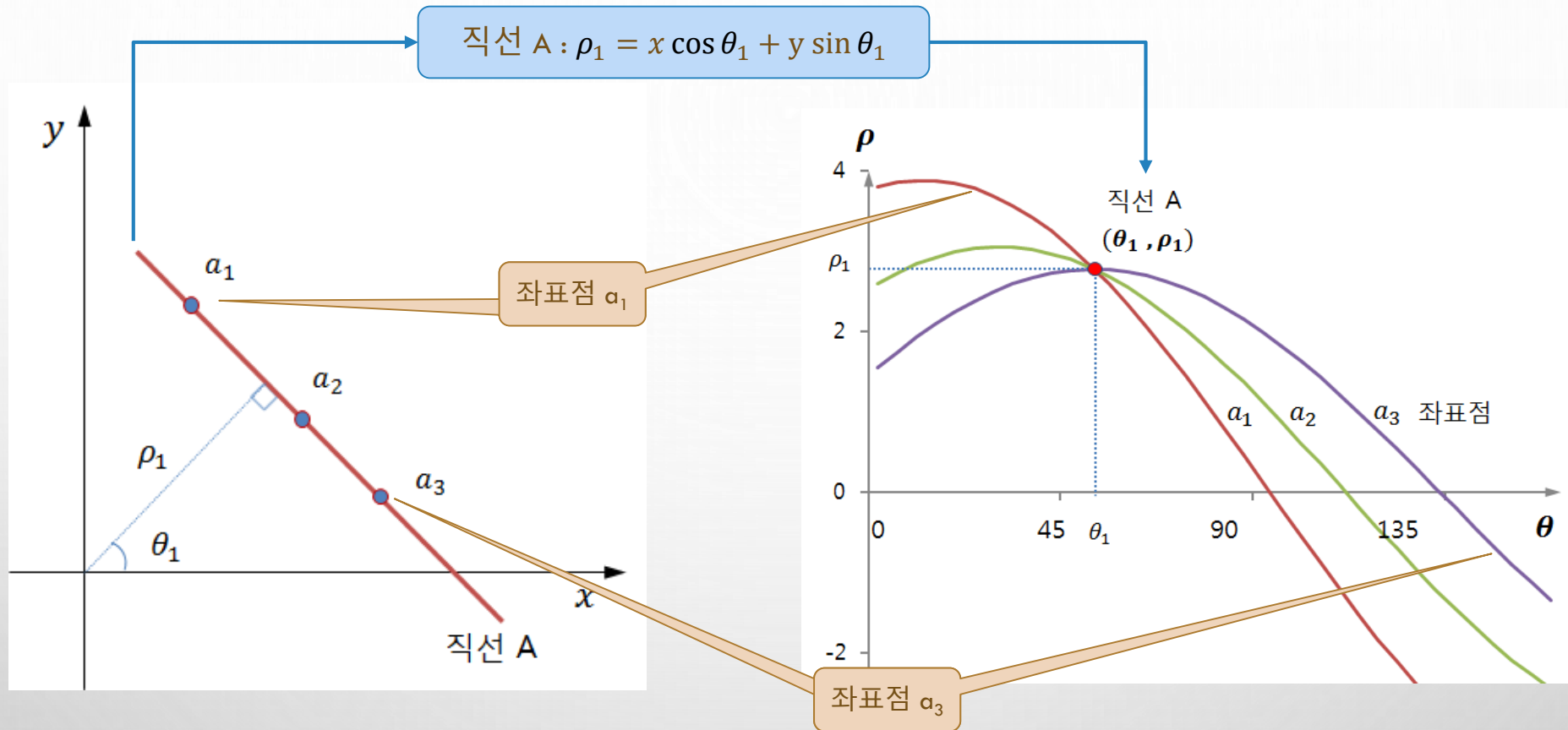
- ❖ 직교 좌표계로 표현되는 영상의 에지 점들을 극 좌표계로 옮겨, 검출하고자 하는 물체의 파라미터(ρ, θ)를 추출하는 방법

$$y = ax + b \leftrightarrow \rho = x \cdot \cos \theta + y \cdot \sin \theta$$

- ❖ 직교 좌표계에서 검출의 문제점 (클래식 허프 변환)

- 수직선일 경우에 기울기가 무한대
- 검출되는 직선의 간격이 동일하지 않아서 검출 속도와 정밀도에서 문제

10.1.1 허프 변환의 좌표계



- ❖ 직교좌표의 직선은 허프변환 좌표에서 한점 (ρ_1, θ_1) 으로 표현
- ❖ 직교좌표의 한 점은 허프변환 좌표에서 곡선으로 표현

10.1.2 허프 변환의 전체 과정

1. 극 좌표계에서 누적 행렬 구성
2. 영상 화소의 직선 검사
3. 직선 좌표에 대한 극좌표 누적 행렬 구성
4. 누적 행렬의 지역 최대값 선정
5. 직선 선별 - 임계값 이상인 누적값 선택 및 정렬

◆ 허프 변환 좌표계를 위한 행렬의 계산

$$\begin{aligned} -\rho_{\max} \leq \rho \leq \rho_{\max}, \quad \rho_{\max} = rows + cols, \quad h = \frac{\rho_{\max}}{\Delta \rho} * 2 \\ 0 \leq \theta < \theta_{\max}, \quad \theta_{\max} = \pi, \quad w = \frac{\pi}{\Delta \theta} \end{aligned}$$

교재 오타 수정

각도간격,
거리간격

10.1.3 허프 누적 행렬 구성

거리간격, 각도간격

누적 행렬

삼각 함수 행렬

직선 극좌표를 누적
행렬의 인덱스로 계산

```
01 def accumulate(image, rho, theta):
02     h, w = image.shape[:2]
03     rows, cols = (h+w) * 2 // rho, int(np.pi / theta)    # 누적행렬 너비, 높이
04     accumulate = np.zeros((rows, cols), np.int32)          # 직선 누적행렬
05
06     sin_cos = [(np.sin(t*theta), np.cos(t*theta)) for t in range(cols)] # 삼각 함수값 저장
07     pts = np.where(image > 0)                                # 넘파이 함수 활용 - 직선좌표 찾기
08
09     polars = np.dot(sin_cos, pts).T                          # 행렬 곱으로 극좌표 계산
10     polars = (polars / rho + rows / 2).astype('int')        # 해상도 변경 및 위치 조정
11
12     for row in polars:
13         for t, r in enumerate(row):                          # 각도, 수직 거리 가져옴
14             accumulate[r, t] += 1                            # 극좌표에 누적
15     return accumulate
```

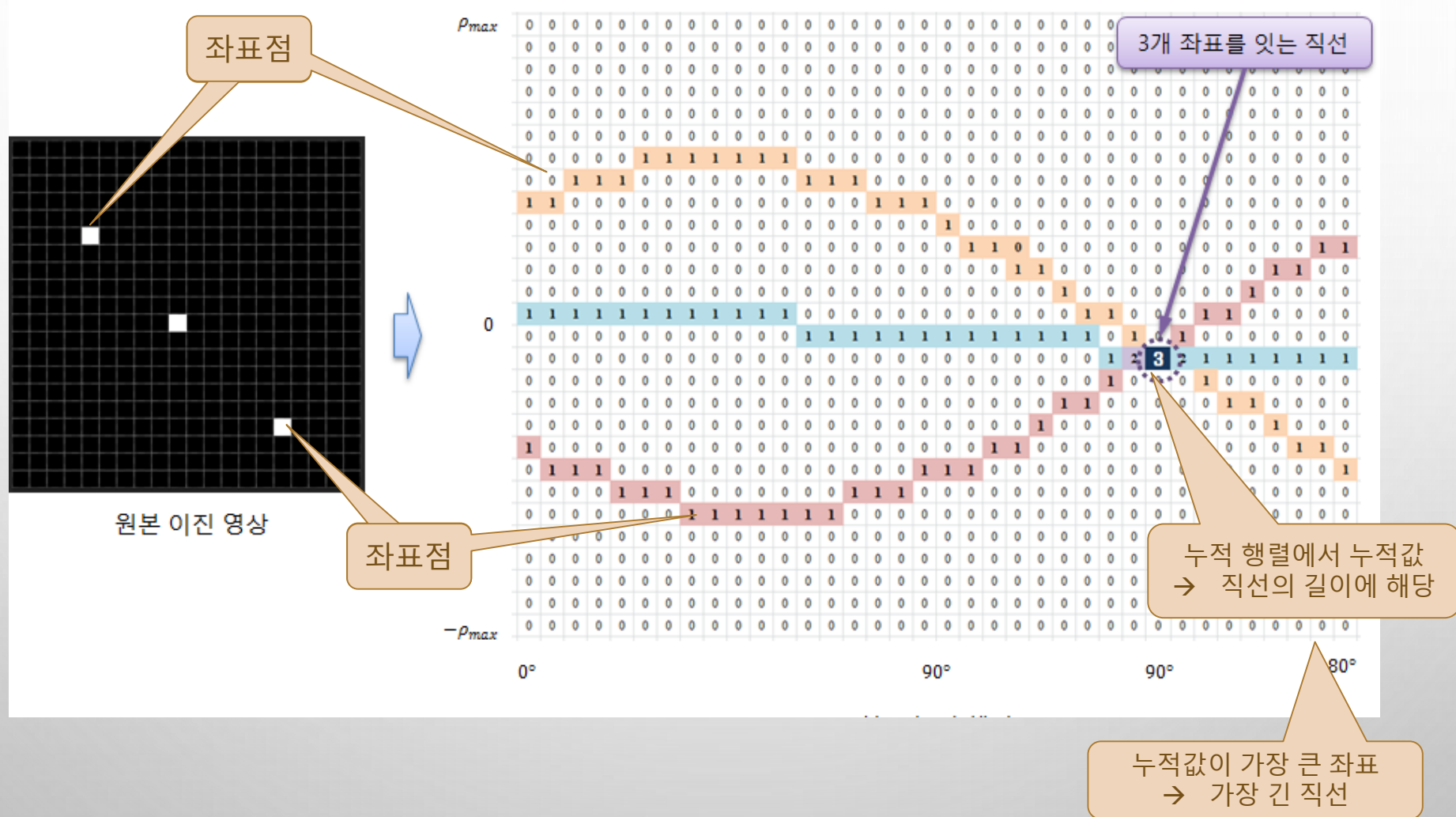
직선 좌표들의 극좌표

0~180도 각도에 대한
삼각함수 값 행렬

$$\begin{bmatrix} \sin \theta_0 & \cos \theta_0 \\ \sin \theta_1 & \cos \theta_1 \\ \sin \theta_2 & \cos \theta_2 \\ \dots & \dots \\ \sin \theta_m & \cos \theta_m \end{bmatrix} \cdot \begin{bmatrix} y_0 & y_1 & y_2 & \dots & y_n \\ x_0 & x_1 & x_2 & \dots & x_n \end{bmatrix} = \begin{bmatrix} \rho_0, \theta_0 & \rho_1, \theta_0 & \rho_2, \theta_0 & \dots & \rho_n, \theta_0 \\ \rho_0, \theta_1 & \rho_1, \theta_0 & \rho_2, \theta_1 & \dots & \rho_n, \theta_1 \\ \rho_0, \theta_2 & \rho_1, \theta_0 & \rho_2, \theta_2 & \dots & \rho_n, \theta_2 \\ \dots & \dots & \dots & \dots & \dots \\ \rho_0, \theta_m & \rho_1, \theta_m & \rho_2, \theta_m & \dots & \rho_n, \theta_m \end{bmatrix}$$

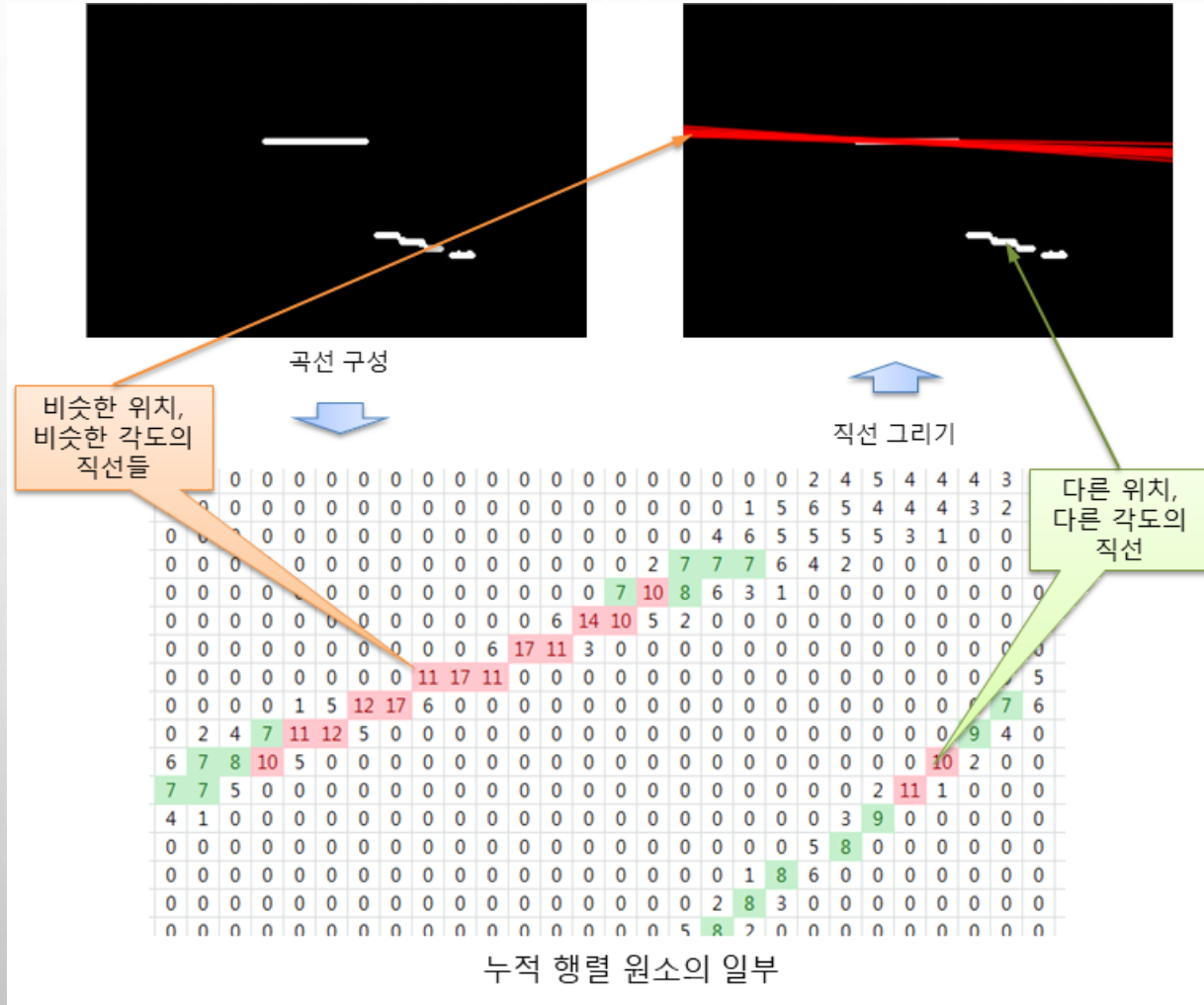
직선 좌표들

10.1.3 허프 누적 행렬 구성



10.1.3 허프 누적 행렬 구성

❖ 한 지점에서 여러 직선 검출의 문제



◆ 마스크를 이용해 지역 최대값 선정



10.1.4 허프 누적 행렬의 지역 최대값 선정

```
01 def masking(accumulate, h, w, thresh):
02     rows, cols = accumulate.shape[:2]
03     rcenter, tcenter = h//2, w//2           # 마스크 크기 절반
04     dst = np.zeros(accumulate.shape, np.uint32)
05
06     for y in range(0, rows, h):
07         for x in range(0, cols, w):
08             roi = accumulate[y:y+h, x:x+w]
09             _, max, _ , (x0, y0) = cv2.minMaxLoc(roi)
10             dst[y+y0, x+x0] = max
11     return dst
```

마스크 크기

누적 행렬 조회

최대값 좌표

결과 행렬 마스크 영역의 최대값
좌표

10.1.5 직선(극 좌표) 선택 및 정렬

❖ 직선 선별 함수

```
01 def select_lines(acc_dst, rho, theta, thresh):
```

```
02     rows = acc_dst.shape[0]
```

임계값이상인
누적 행렬의 좌표(ρ, θ)

```
03     r, t = np.where(acc_dst>thresh)
```

임계값 이상 인덱스 가져옴

```
05     rhos = ((r - (rows / 2)) * rho)
```

인덱스로 수직 거리 계산

```
06     radians = t * theta
```

인덱스로 각도 계산

```
07     values = acc_dst[r, t]
```

인덱스로 누적값 가져옴

누적값 기준 정렬
인덱스

```
08     idx = np.argsort(values)[::-1]
```

내림차순 정렬 인덱스

```
10     lines = np.transpose([rhos, radians])
```

리스트 전치하여 행렬 생성

```
11     lines = lines[idx, :]
```

누적값 기준으로 극좌표 정렬

```
12
```

```
13     return np.expand_dims(lines, axis=1)
```

1번(열) 차원 증가

OpenCV 함수와 동일하
게 반환 자료형 맞춤

수직거리 행렬

$$\begin{bmatrix} \rho_0 & \rho_1 & \rho_2 & \cdots & \rho_n \\ \theta_0 & \theta_1 & \theta_2 & \cdots & \theta_n \end{bmatrix}$$

각도 행렬

직선 극 좌표

전치

$$\begin{bmatrix} \rho_0 & \theta_0 \\ \rho_1 & \theta_1 \\ \rho_2 & \theta_2 \\ \vdots & \vdots \\ \rho_n & \theta_n \end{bmatrix}$$

직선 극 좌표

10.1.7 최종 완성 프로그램

허프 변환
전체 과정 수행 함수

직선상의 2개 좌표 계산

```

01 import numpy as np, cv2, math
02 from Common.hough import accumulate, masking, select_lines    # 허프 변환 함수 импорт
03
04 def houghLines(src, rho, theta, thresh):                        # 허프 변환 함수
05     acc_mat = accumulate(src, rho, theta)                      # 직선 누적 행렬 계산
06     acc_dst = masking(acc_mat, 7, 3, thresh)                   # 마스킹 처리 - 7행, 3열
07     lines = select_lines(acc_dst, rho, theta, thresh)          # 임계 직선 선택
08     return lines
09
10 def draw_houghLines(src, lines, nline):                        # 검출 직선 그리기 함수
11     dst = cv2.cvtColor(src, cv2.COLOR_GRAY2BGR)               # 컬러 영상 변환
12     min_length = min(len(lines), nline)
13
14     for i in range(min_length):
15         rho, radian = lines[i, 0, 0:2]                         # 수직 거리, 각도 - 3차원 행렬임
16         a, b = math.cos(radian), math.sin(radian)
17         pt= (a * rho, b * rho)                                  # 검출 직선상의 한 좌표
18         delta= (-1000 * b, 1000 * a)                           # 직선상의 이동 위치
19         pt1 = np.add(pt, delta).astype('int')
20         pt2 = np.subtract(pt, delta).astype('int')
21         cv2.line(dst, tuple(pt1), tuple(pt2), (0, 255, 0), 2, cv2.LINE_AA)
22
23     return dst
24

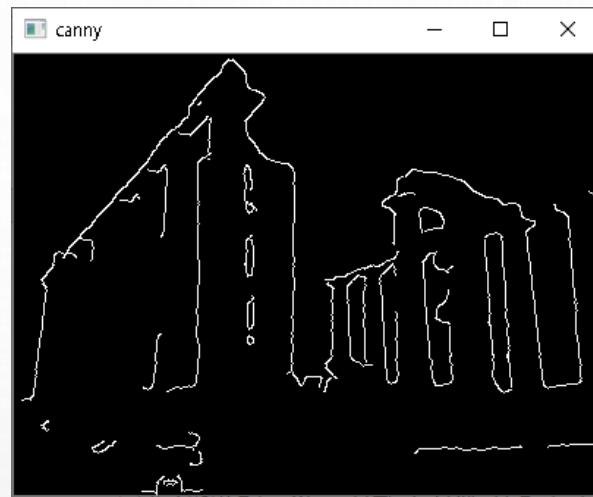
```


10.1.7 최종 완성 프로그램

```
25 image = cv2.imread("images/hough.jpg", cv2.IMREAD_GRAYSCALE)
26 if image is None: raise Exception("영상파일 읽기 에러")
27 blur = cv2.GaussianBlur(image, (5, 5), 2, 2)           # 가우시안 블러링
28 canny = cv2.Canny(blur, 100, 200, 5)                  # 캐니 에지 추출
29
30 rho, theta = 1, np.pi / 180                          # 수직거리 간격, 각도 간격
31 lines1 = houghLines(canny, rho, theta, 80)             # 저자 구현 함수
32 lines2 = cv2.HoughLines(canny, rho, theta, 80)         # OpenCV 함수
33 dst1 = draw_houghLines(canny, lines1, 7)               # 직선 그리기
34 dst2 = draw_houghLines(canny, lines2, 7)
35
36 cv2.imshow("image", image)
37 cv2.imshow("canny", canny);
38 cv2.imshow("detected lines", dst1)
39 cv2.imshow("detected lines_OpenCV", dst2)
40 cv2.waitKey(0)
```

10.17 최종 완성 프로그램

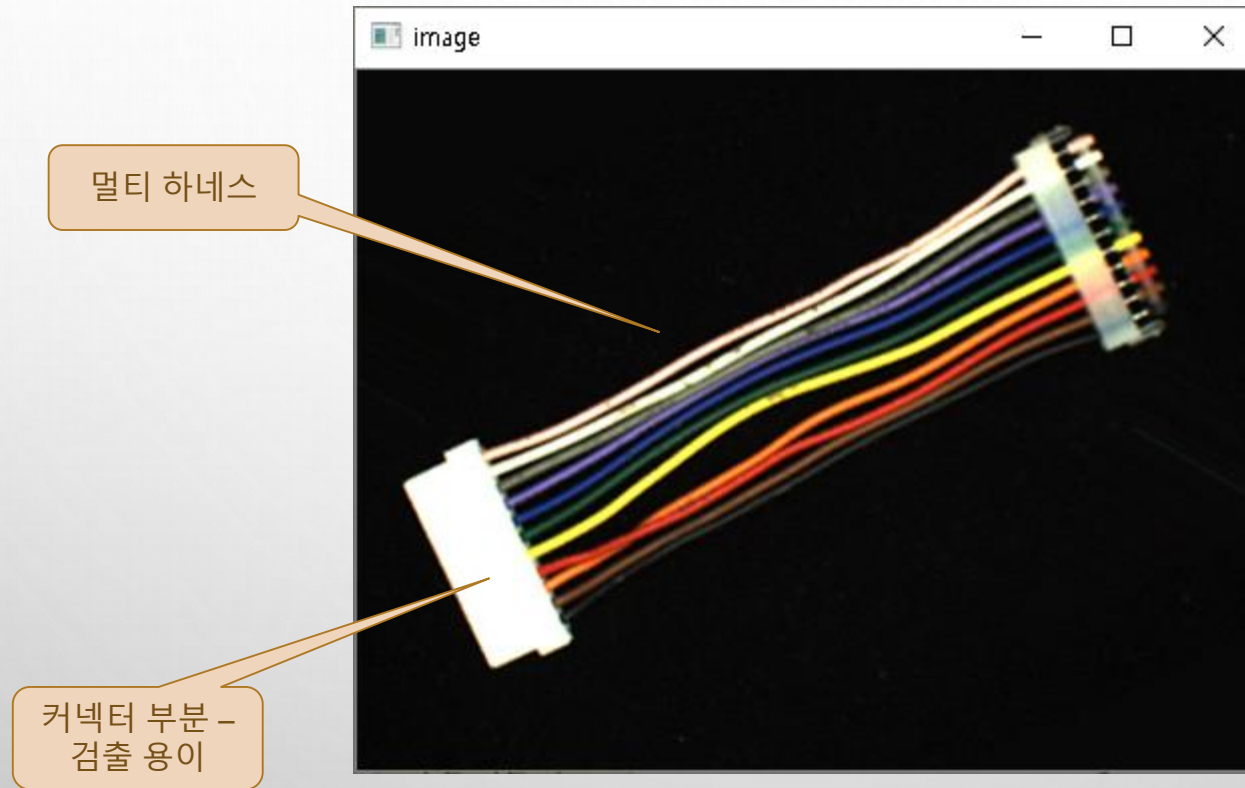
◆ 실행결과



10.1.8 심화예제 -멀티 하네스의 전처리(기울기 보정)

◆ 멀티하네스란

- ❖ 전자제품 내부에서 다종의 전선들을 한 번에 연결시키는 케이블 커넥터



10.1.8 심화예제 -멀티 하네스의 전처리(기울기 보정)

심화예제 10.1.2 직선 검출을 이용한 멀티 하네스 기울기 보정 - 02.correct_object.py

```
01 import numpy as np, cv2
02 from Common.hough import *           # 허프 변환 관련 사용자 정의 함수 포함
03
04 def detect_maxObject(img):           객체 외곽선 검출
05     results = cv2.findContours(img, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
06     if int(cv2.__version__[0]) >= 4: # OpenCV 4.x - 2 원소 튜플 반환
07         contours = results[0]
08     else:
09         contours = results[1]         # OpenCV 3.x - 3 원소 튜플 반환
10
11     areas = [cv2.contourArea(c) for c in contours]
12     idx = np.argsort(areas)           # 오름차순 정렬 인덱스
13     max_rect = contours[idx[-1]]      # 오름차순이라 마지막 원소가 최대
14
15     rect = cv2.boundingRect(max_rect)
16     rect = np.add(rect, (-10, -10, 20, 20))
17     return rect
18
```

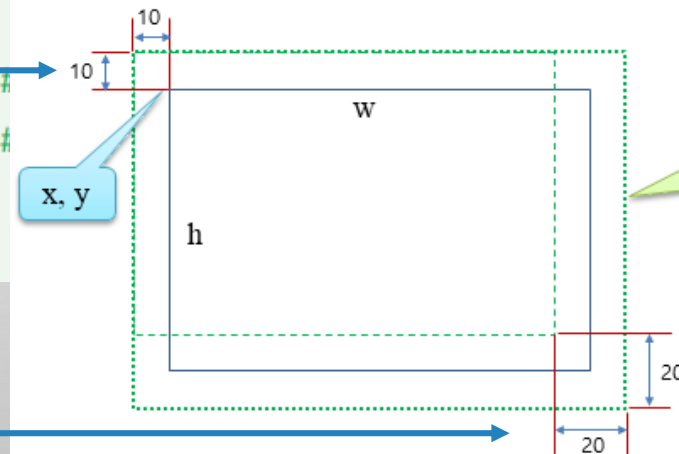
검출 객체 넓이들에 대한 정렬 인덱스

객체 외곽선 검출

검출 객체들의 넓이 가져옴

x, y

상하좌우 10픽셀 확대 사각형



심화예제 – 멀티하네스 기울기 보정

```
19 image = cv2.imread("images/harness.jpg", cv2.IMREAD_COLOR)
20 if image is None: raise Exception("영상파일 읽기 에러")
21
22 rho, theta = 1, np.pi / 180 # 허프 변환 거리&각도 간격
23 gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY) # 명암도 영상 변환
24 th_gray = cv2.threshold(gray, 240, 255, cv2.THRESH_BINARY)[1] # 이진 영상 변환
25 kernel = np.ones((3, 3), np.uint8)
26 morph = cv2.erode(th_gray, kernel, iterations=2) # 침식 연산- 2번 반복
27
28 x, y, w, h = detect_maxObject(np.copy(morph)) # 가장 큰 객체 rect 검출
29 roi = th_gray[y:y+h, x:x+w]
30
31 canny = cv2.Canny(roi, 40, 100) # 캐니 에지 검출
32 lines = houghLines(canny, rho, theta, 50) # 허프 직선 검출
33 # lines = cv2.HoughLines(canny, rho, theta, 50) # OpenCV 함수
34
35 cv2.rectangle(morph, (x, y, w, h), 100, 2) # 큰 객체 사각형 표시
36 canny_line = draw_houghLines(canny, lines, 1) # 직선 표시
37
```

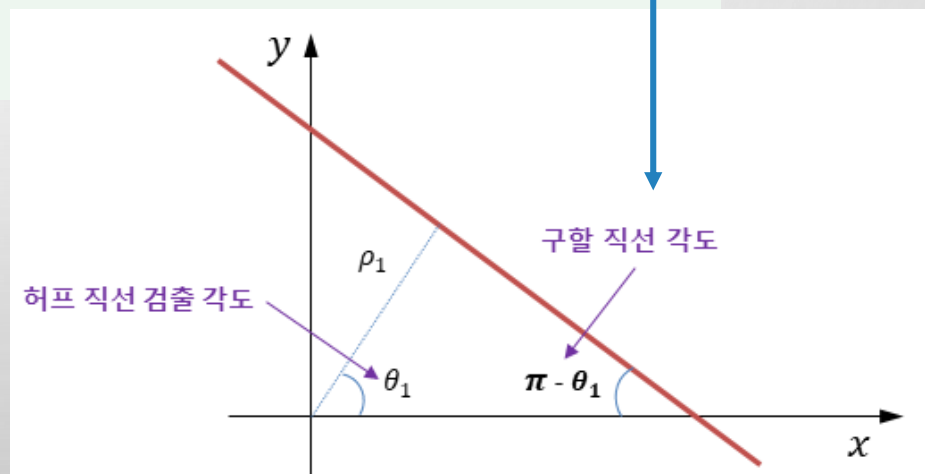
검출 객체 관심영역

배경 잡음 및
케이블 부분 제거

직선 기울기 이용해
커넥터 기울기 계산

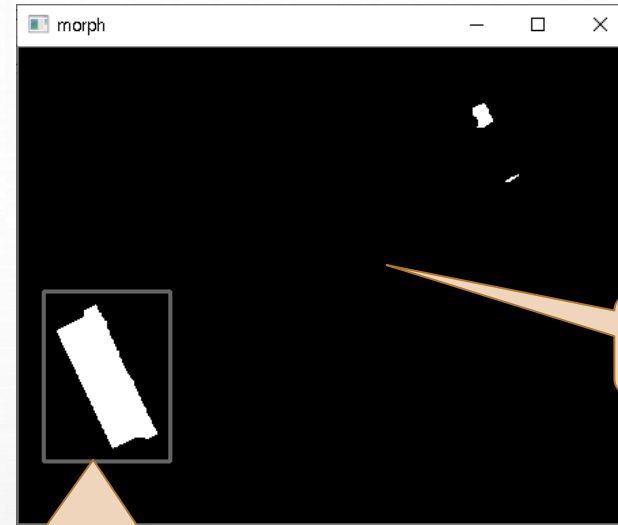
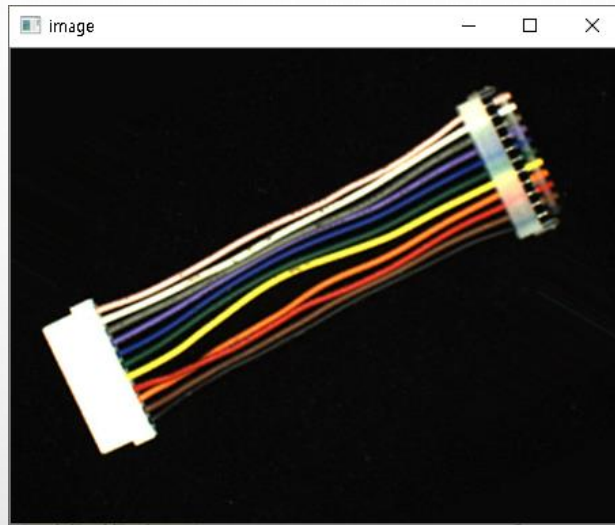
심화예제 – 멀티하네스 기울기 보정

```
38 angle = (np.pi - lines[0, 0, 1]) * 180 / np.pi      # 회전 각도 계산
39 h, w = image.shape[:2]
40 center = (w//2, h//2)                                # 입력 영상의 중심점
41 rot_map = cv2.getRotationMatrix2D(center, -angle, 1)  # 역방향 회전 행렬 계산
42 dst = cv2.warpAffine(image, rot_map, (w, h), cv2.INTER_LINEAR) # 역회전 수행
43
44 cv2.imshow("image", image)
45 cv2.imshow("morph", morph)
46 cv2.imshow("line_detect", canny_line)
47 cv2.resizeWindow("line_detect", 250, canny_line.shape[0])
48 cv2.imshow("dst", dst)
49 cv2.waitKey(0)
```



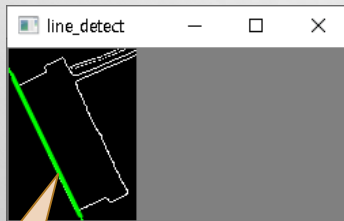
심화예제 – 멀티하네스 기울기 보정

◆ 실행결과

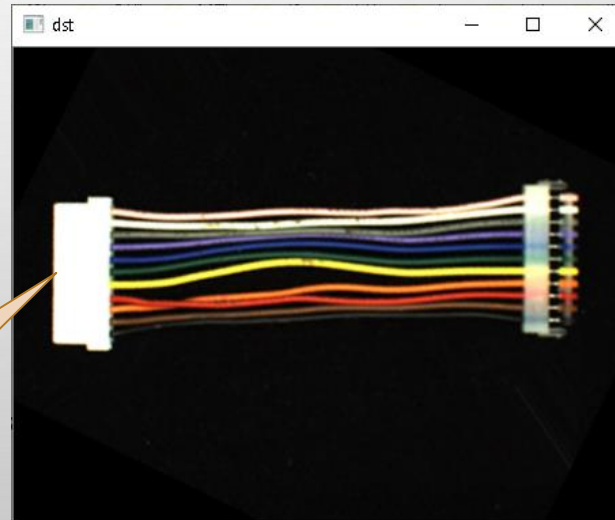


침식 연산으로
케이블 부분 제거

가장 큰 객체 검출 -
멀티 하네스의 커넥터 부분



허프변환 이용
직선 검출



직선 기울기 이용
기울기 보정

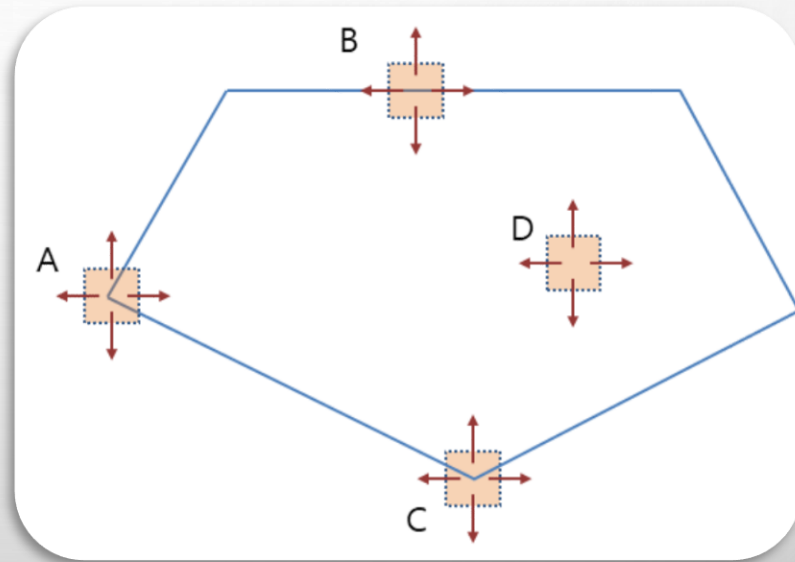
10.2 코너 검출

◆ 에지나 직선

- ❖ 영상 구조 파악 및 객체 검출에는 도움이 되지만, 영상 매칭에는 큰 도움이 되지 않음
- ❖ 에지 강도와 방향 정보만 가지므로 영상 매칭하기엔 정보 부족

◆ 꼭지점 혹은 코너(corner)

- ❖ 영상의 밝기 변화
 - A, C - 모든 방향에서 밝기변화 큼
 - B, D - 밝기 변화가 상대적으로 적음



10.2 코너 검출

◆ 모라벡(Moravec)

❖ 영상 변화량(SSD: Sum of Squared Difference)

$$E(u, v) = \sum_y \sum_x w(x, y) \cdot (I(x+u, y+v) - I(x, y))^2$$

- 현재 화소에서 u, v 방향으로 이동했을 때의 밝기 변화량의 제곱
- (u, v) 를 $(1, 0), (1, 1), (0, 1), (-1, 1)$ 의 4개 방향으로 한정

◆ 문제점

- ❖ 0과 1의 값만 갖는 이진 윈도우 사용으로 노이즈에 취약
- ❖ 4개 방향으로 한정시켰기 때문에 45도 간격의 에지만 고려

10.2 코너 검출

◆ 해리스(Harris) 검출

- ❖ 이진 윈도우 $\omega(u, v)$ 대신에 점진적으로 변화하는 가우시안 마스크 $G(x, y)$ 적용

$$E(u, v) = \sum_y \sum_x G(x, y) \cdot (I(x+u, y+v) - I(x, y))^2$$

- ❖ 모든 방향에서 검출할 수 있도록 미분 도입

$$\begin{aligned} I(x+u, y+v) &\cong I(x, y) + v d_y(x, y) + u d_x(x, y) \\ E(u, v) &\cong \sum_y \sum_x G(x, y) \cdot (v d_y(x, y) + u d_x(x, y))^2 \end{aligned}$$

$$\begin{aligned} E(u, v) &\cong \sum_y \sum_x G(x, y) \cdot (v d_y + u d_x)^2 \\ &= \sum_y \sum_x G(x, y) \cdot (v^2 d_y^2 + u^2 d_x^2 + 2v u d_x d_y) \\ &= \sum_y \sum_x G(x, y) \cdot (u \ v) \begin{pmatrix} d_x^2 & d_x d_y \\ d_x d_y & d_y^2 \end{pmatrix} \begin{pmatrix} u \\ v \end{pmatrix} \\ &= (u \ v) M \begin{pmatrix} u \\ v \end{pmatrix}, \quad M = \sum_y \sum_x G(x, y) \begin{pmatrix} d_x^2 & d_x d_y \\ d_x d_y & d_y^2 \end{pmatrix} \end{aligned}$$

특징 가능성을 직접 계산하는 대신 행렬 M 의 식으로 정리

10.2 코너 검출

- ❖ 행렬 M 에서 고유벡터를 구하면 경계선 방향에 수직인 벡터 두 개 얻음
- ❖ 행렬 M 의 고유값(λ_1, λ_2)으로 코너 응답 함수 계산

$$R = \lambda_1, \lambda_2 - k \cdot (\lambda_1 + \lambda_2)^2$$

- ❖ 고유값 대신 행렬식(det)과 대각합(trace)을 통해 코너 응답 함수로 이용
 - 고유값 계산은 고유값 분해의 복잡한 과정 거침

$$M = \begin{pmatrix} d_x^2 & d_x d_y \\ d_x d_y & d_y^2 \end{pmatrix} = \begin{pmatrix} a & c \\ c & b \end{pmatrix}$$

$$R = \det(M) - k \cdot \text{trace}(M)^2 = (ab - c^2) - k \cdot (a + b)^2$$

◆ 해리스 코너 검출 특징

- ❖ 영상의 평행이동, 회전 변환에는 불변(invariant)하는 특징
- ❖ 어파인(affine) 변환이나 조명(illumination) 변화에도 어느 정도 강인성

10.2 코너 검출

◆ 전체 과정

1. 소벨 마스크로 미분 행렬 계산 (dx, dy)
2. 미분 행렬의 곱 계산 (dx^2, dy^2, dxy)
3. 곱 행렬에 가우시안 마스크 적용
4. 코너 응답함수 $C = \det(M) - k \cdot \text{trace}(M)^2$ 계산
5. 비최대치 억제

10.2 코너 검출

◆ 소스 코드

예제 10.2.1 헤리스 코너 검출 - 03.harris_dectect.py

```
01 import numpy as np, cv2
02 from Common.utils import put_string          # 영상에 글쓰기 함수 импорт
03
04 def cornerHarris(image, ksize, k):           # 해리스 코너 검출 함수
05     dx = cv2.Sobel(image, cv2.CV_32F, 1, 0, ksize) # 미분 행렬-수평 소벨 마스크
06     dy = cv2.Sobel(image, cv2.CV_32F, 0, 1, ksize) # 미분 행렬-수직 소벨 마스크
07
08     a = cv2.GaussianBlur(dx * dx, (5, 5), 0)      # 가우시안 블러링 수행
09     b = cv2.GaussianBlur(dy * dy, (5, 5), 0)
10     c = cv2.GaussianBlur(dx * dy, (5, 5), 0)
11
12     corner = (a * b - c**2) - k * (a + b)**2      # 코너 응답 함수-행렬 연산 적용
13     return corner
```

곱 행렬 계산
(dx^2 , dy^2 ,
 dxy)

10.2 코너 검출

◆ 소스 코드

```
15 def drawCorner(corner, image, thresh):
16     corner = cv2.normalize(corner, 0, 300, cv2.NORM_MINMAX, dtype=cv2.CV_8U)
17     pts = np.where(corner > thresh)
18     h,w = corner.shape
19     corners = []
20     for j, i in np.transpose(pts):
21         if 0 < i < h-1 and 0 < j < w-1:
22             neighbor = corner[i-1:i+2, j-1:j+2].flatten()
23             max = np.max(neighbor[1::2])
24             if corner[i, j] >= max: corners.append((j, i))
25
26     for pt in corners:
27         cv2.circle(image, pt, 3, (0, 230, 0), -1) # 좌표 표시/
28     print("임계값: %2d , 코너 개수: %2d" %(thresh, len(corners)))
29     return image
```

반지름 3인
녹색 원 표시

10.2 코너 검출

◆ 소스 코드

```
30 def onCornerHarris(thresh):  
31     img1 = drawCorner(corner1, np.copy(image), thresh)  
32     img2 = drawCorner(corner2, np.copy(image), thresh)  
33  
34     put_string(img1, "USER", (10, 30), "")  
35     put_string(img2, "OpenCV", (10, 30), "")  
36     dst = cv2.repeat(img1, 1, 2)  
37     dst[:, img1.shape[1]:, :] = img2  
38     cv2.imshow("harris detect", dst)  
39  
40 image = cv2.imread("images/harris.jpg", cv2.IMREAD_COLOR)  
41 if image is None: raise Exception("영상파일 읽기 에러")  
42  
43 blockSize = 4  
44 apertureSize = 3  
45 k = 0.04  
46 thresh = 2  
47 gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)  
48 corner1 = cornerHarris(gray, apertureSize, k)  
49 corner2 = cv2.cornerHarris(gray, blockSize, apertureSize, k)  
50  
51 onCornerHarris(thresh)  
52 cv2.createTrackbar("Threshold", "harris detect", thresh, 20, onCornerHarris)  
53 cv2.waitKey(0)
```

트랙바 콜백 함수 - 이벤트시마다 영상에 코너 표시

트랙바 콜백 함수

영상에 문자표시 함수 호출

두 개 영상을 한 윈도우 표시

결과 영상 오른쪽 영역에 OpenCV 함수 결과 영상

이웃 화소 범위

소벨 마스크 크기

코너 응답 임계값

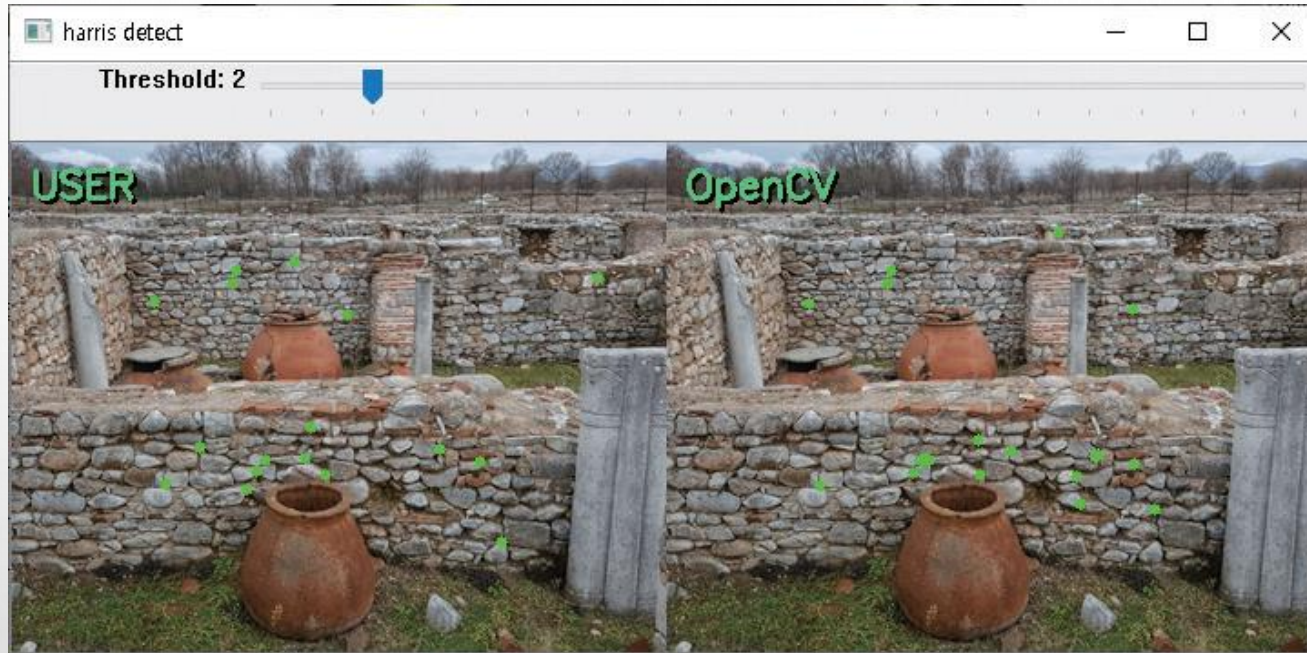
사용자 정의 함수

OpenCV 제공 함수

트랙바 콜백 함수 등록

10.2 코너 검출

◆ 실행결과



10.3 k-최근접 이웃 분류기

- ◆ 10.3.1 k-최근접 이웃 분류기의 이해
- ◆ 10.3.2 k-NN을 위한 KNearest 클래스의 이해
- ◆ 10.3.3 k-NN 응용

10.3.1 k-최근접 이웃 분류기의 이해

◆ 최근접 이웃 알고리즘

- ❖ 기존에 가지고 있는 데이터들을 일정한 규칙에 의해 분류된 상태에서 새로운 입력 데이터의 종류를 예측하는 분류 알고리즘
- ❖ 학습 클래스의 샘플들과 새 샘플의 거리가 가장 가까운(nearest)클래스로 분류
 - ‘가장 가까운 거리’
 - ✓ 미지의 샘플과 학습 클래스 샘플간의 유사도가 가장 높은 것을 의미
 - ✓ 유클리드 거리(euclidean distance), 해밍 거리(hamming distance), 차분 절대값

10.3.1 k-최근접 이웃 분류기의 이해

◆ k-최근접 이웃 분류(k-Nearest Neighbors: k-NN)

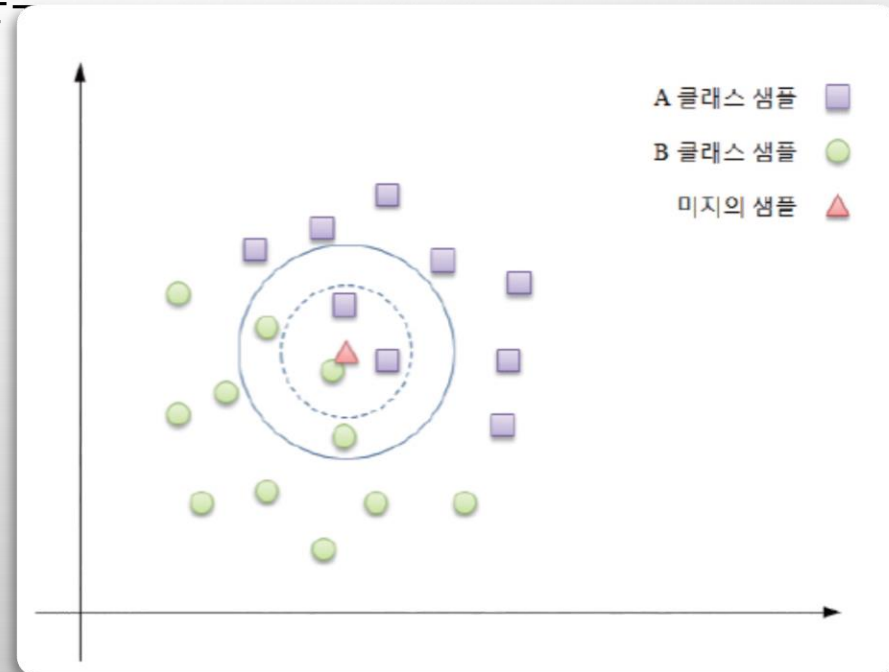
- ❖ 학습된 클래스들에서 여러개(k)의 가까운 이웃 선출
- ❖ 이를 이용해 미지의 샘플들을 분류하는 방법

❖ k가 3일 경우

- 미지 샘플 주변 가장 가까운 이웃 3개 선출
- 이 중 많은 수의 샘플을 가진 클래스로 미지의 샘플 분류
- A클래스 샘플 2개, B클래스 샘플 1개 → A클래스 분류

❖ k가 5일 경우

- 실선 큰 원내에 있는 가장 가까운 이웃 5개 선출
- 2개 A 클래스, 3개 B 클래스 → B클래스로 분류



10.3.2 k-NN을 위한 KNearest 클래스의 이해

예제 10.3.1

임의 좌표 생성 - 04.kNN_exam.py

```
01 import numpy as np, cv2
02
03 def draw_points(image, group, color):
04     for p in group:
05         pt = tuple(p.astype(int))
06         cv2.circle(image, pt, 3, color, cv2.FILLED)
07
08 nsample = 50
09 traindata = np.zeros((nsample*2, 2), np.float32)
10 label = np.zeros((nsample*2, 1), np.float32)
11
12 cv2.randn(traindata[:nsample], 150, 30)
13 cv2.randn(traindata[nsample:], 250, 60)
14 label[:nsample], label[nsample:] = 0, 1
15
16 K = 7
17 knn = cv2.ml.KNearest_create()
18 knn.train(traindata, cv2.ml.ROW_SAMPLE, classlable)
19
```

반지름 3의 원 표시 - 점으로 표시

정수 원소 튜플

반환 행렬

그룹당 학습 데이터 수

전체 학습 데이터 행렬

레이블 행렬 생성

행렬 원소에 정규분포를
따르는 임의값 지정

정규분포 랜덤 값 생성

레이블 기준값 지정

두 그룹에 레이블 값 지정

kNN 클래스로 객체 생성

학습 수행

10.3.2 k-NN을 위한 KNearest 클래스의 이해

```
20 points = [(x, y) for y in range(400) for x in range(400) ]      # 검사 좌표 리스트 생성
21 ret, resp, neig, dist = knn.findNearest(np.array(points, np.float32), K)  # 분류 수행
22
23 colors = [(0, 180, 0) if p else (0, 0, 180) for p in resp]        # 분류된 좌표들에 색지정
24 image = np.reshape(colors, (400, 400, 3)).astype('uint8')        # 3채널 컬러
25
26 draw_points(image, traindata[:nsample], color=(0, 0, 255))        # 좌표들을 400x400 크기
27 draw_points(image, traindata[nsample:], color=(0, 255, 0))        # 영상으로 형태 변경
28 cv2.imshow("sample K="+ str(K), image)
29 cv2.waitKey(0)
```

분류된 좌표들에 색지정

좌표들을 400x400 크기
영상으로 형태 변경

K를 7로 지정하여 분류 수행 결과

K를 15로 지정하여 분류 수행 결과

학습데이터들

학습데이터들

영상 좌표들에 분류 수행
- 분류결과를 색상으로 표시

◆ 실행결과

10.3.3 MNIST 데이터 사용

◆ MNIST(Modified National Institute of Standards and Technology) 데이터셋

- ❖ 필기 숫자 영상으로 구성된 대형 데이터베이스
- ❖ 다양한 영상처리 시스템을 학습하기 위해 일반적으로 사용
- ❖ 다운로드 <http://deeplearning.net/data/mnist/mnist.pkl.gz>



10.3.3 MNIST 데이터 사용

예제 10.3.2

MNIST 데이터 다운로드 및 시각화 - 05.mnist_kNN_train.py

```
01 import cv2, numpy as np
02 import pickle, gzip, os
03 from urllib.request import urlretrieve
04 import matplotlib.pyplot as plt
05
06 def load_mnist(filename):
07     if not os.path.exists(filename):
08         print("Downloading")
09         link = "http://deeplearning.net/data/mnist/mnist.pkl.gz"
10         urlretrieve(link, filename)
11     with gzip.open(filename, 'rb') as f:
12         return pickle.load(f, encoding='latin1')
13
14 def graph_image(data, lable, title, nsample):
15     plt.figure(num=title, figsize=(10, 10))
16     rand_idx = np.random.choice(range(data.shape[0]), nsample)
17     for i, id in enumerate(rand_idx):
18         img = data[id].reshape(28, 28)
19         plt.subplot(6, 4, i+1), plt.axis('off'), plt.imshow(img, cmap='gray')
20         plt.title("%s: %d" % (title, lable[id]))
21     plt.tight_layout()
22
```

압축해제위한 모듈

웹사이트 링크 다운로드 함수

MNIST 데이터셋 다운로드 함수

현재 폴더에 파일 없으면 다운

mnist 데이터셋 다운주소

다운로드

pickle 모듈로 파일에서 로드함

데이터 시각화 함수

전체 데이터중 랜덤하게 nsample개 선택

데이터 번호 랜덤 생성

1행 행렬을 영상 행태로 변경

서브플롯 타이틀

10.3.3 MNIST 데이터 사용

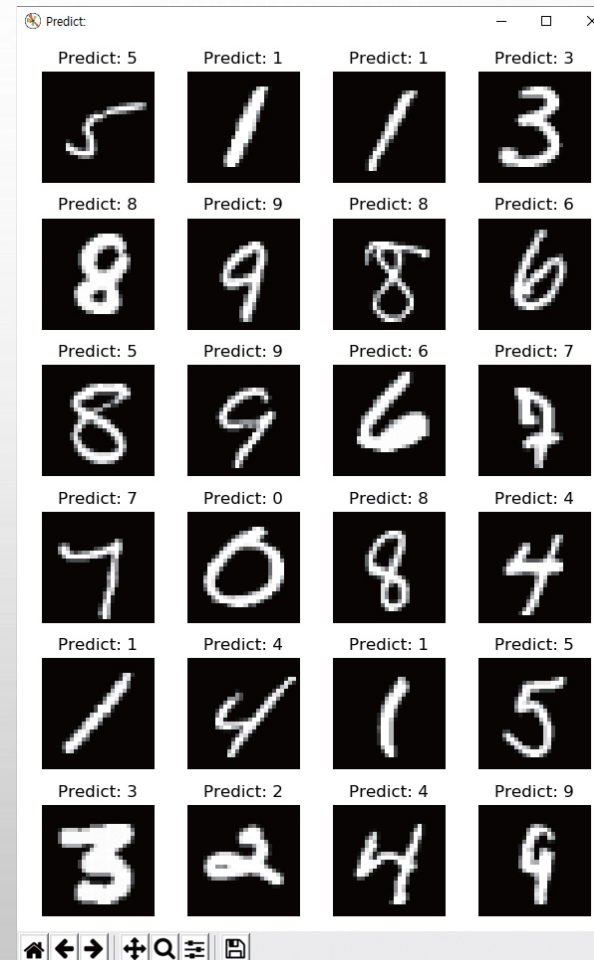
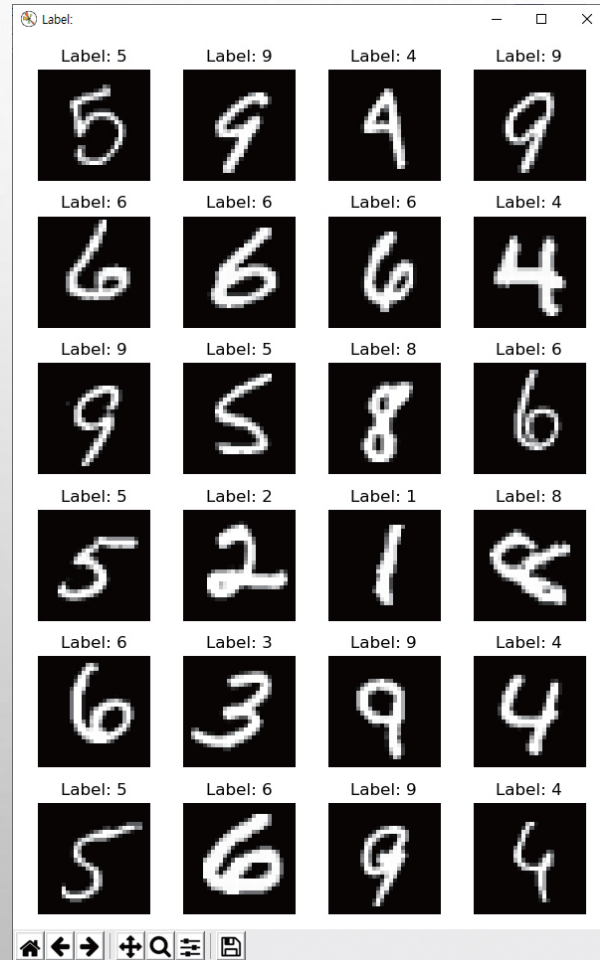
오타
수정

```
23 train_set, valid_set, test_set = load_mnist("mnist.pkl.gz") # MNIST 데이터셋 다운
24 train_data, train_label = train_set
25 test_data, test_label = test_set
26 ## MNIST 로드 데이터 크기 확인
27 print("train_set=", train_set[0].shape) # 학습 데이터셋
28 print("valid_set=", valid_set[0].shape) # 검증 데이터셋
29 print("test_set=", test_set[0].shape) # 테스트 데이터셋
30
31 print("training...")
32 knn = cv2.ml.KNearest_create() # knn 객체 생성
33 knn.train(train_data, cv2.ml.ROW_SAMPLE, train_label) # k-NN 학습 수행
34
35 nsample = 100 # 1행이 하나의 샘플
36 print("%d 개 predicting..." % nsample)
37 _, resp, _ = knn.findNearest(test_data[:nsample], k=5) # k-NN 분류 수행
38 accur = sum(resp.flatten() == test_label[:nsample]) # 분류 정확도 측정
39
40 print('정확도=', accur / nsample * 100, '%') # 분류 결과가 2차원 행렬이기
41 graph_image(train_data, train_label, 'label', 24) # 데이터 영상으로 그리기
42 graph_image(test_data[:nsample], resp, 'predict', 24)
43 plt.show() # 2개 그림 동시에 열기
```


10.3.3 MNIST 데이터 사용

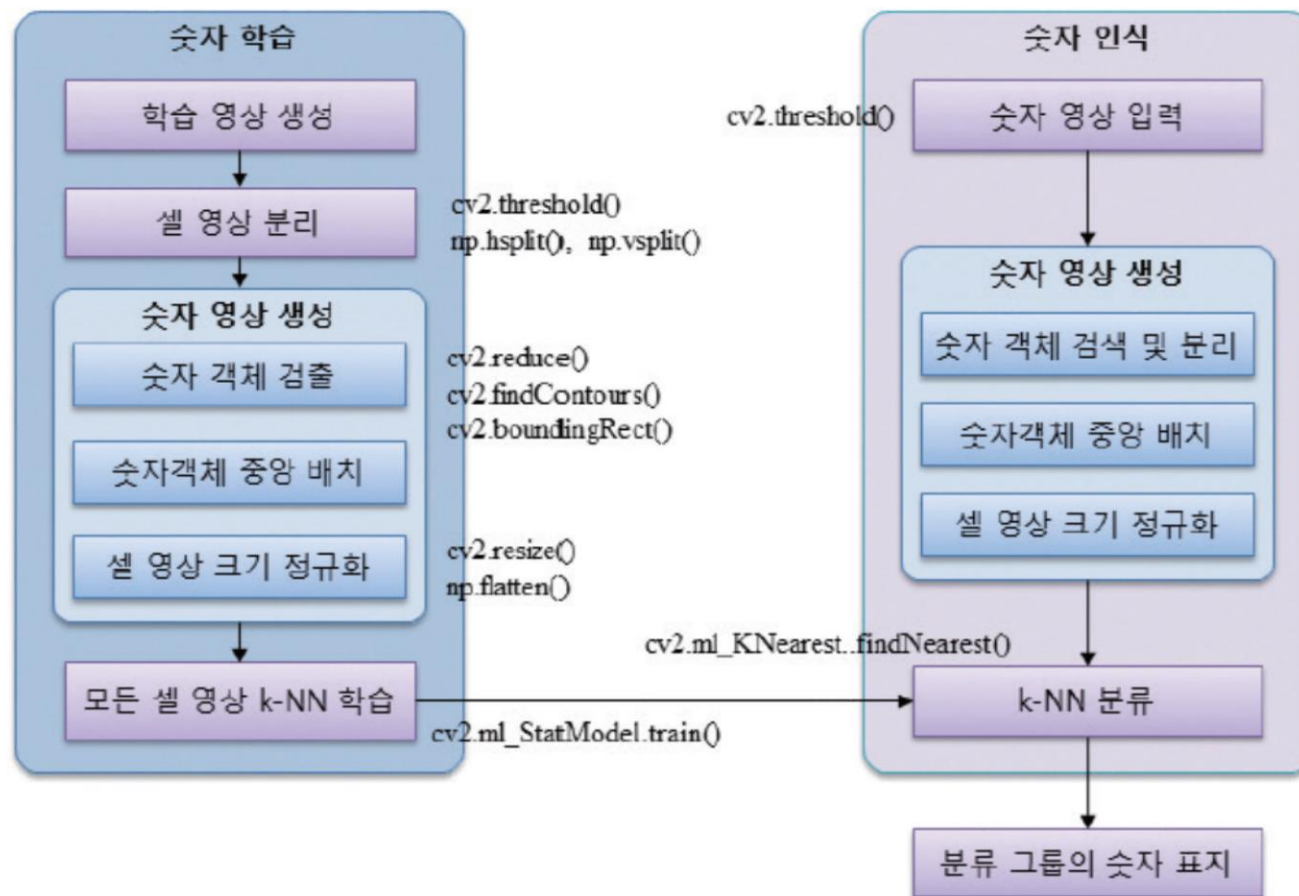
◆ 실행결과

```
Run: 05.mnist_kNN_train
C:\Python\python.exe D:/source/chap10/05.mnist_kNN_train.py
train_set= (50000, 784)
valid_set (10000, 784)
test_set (10000, 784)
training...
100개 predicting...
정확도= 98.0 %
```



10.3.4 k-NN 응용

◆ k-NN을 이용해서 숫자 영상에서 숫자 인식

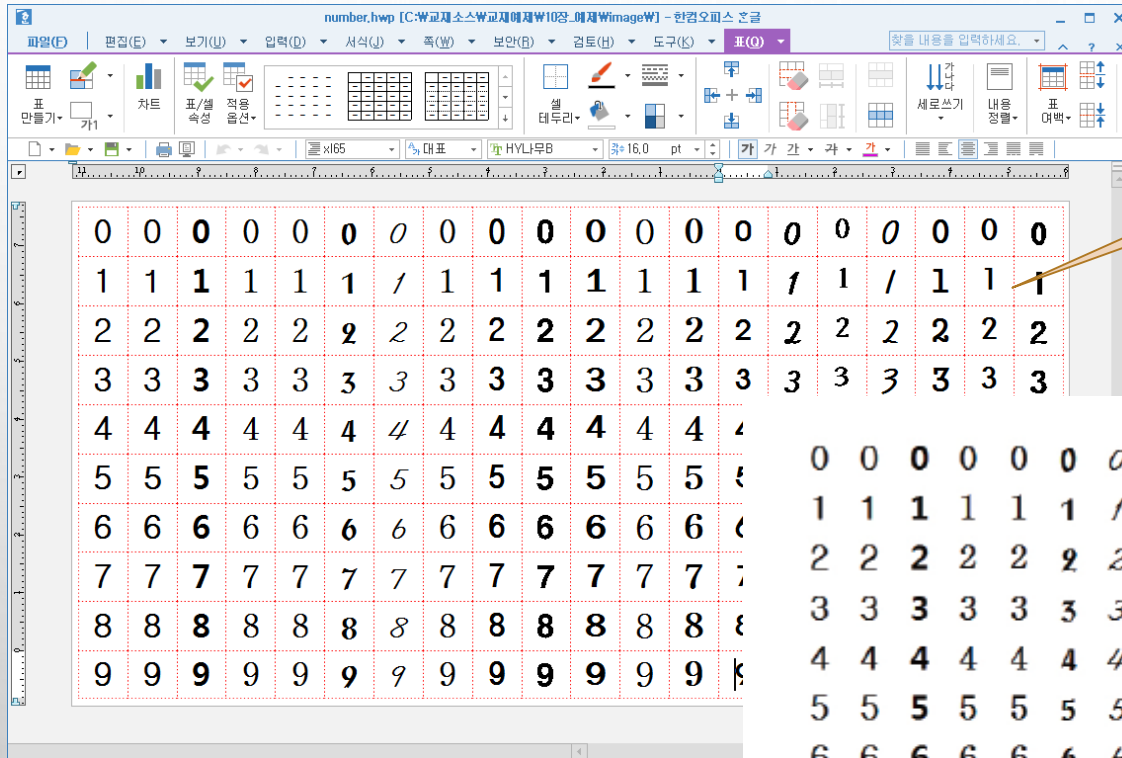


〈그림 10.3.2〉 숫자 인식 예제 전체 수행 과정

1) 학습 영상의 생성

◆ 학습을 위한 숫자 영상

❖ 한글 파일로 작성 : 'chap10/image/numbers.hwp'



train_numbers.png
파일로 저장

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3
4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4
5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5
6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6
7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7
8	8	8	8	8	8	8	8	8	8	8	8	8	8	8	8	8	8	8	8	8
9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9

〈그림 10.3.3〉 학습을 위한 숫자 영상의 예

2) 학습 영상에서 셀 영상 분리

◆ 전체 숫자 영상에서 각 숫자 영상의 셀 영상으로 분리

예제 10.3.3

MNIST 데이터 다운로드 및 시각화 - 06.cells_visibility.py

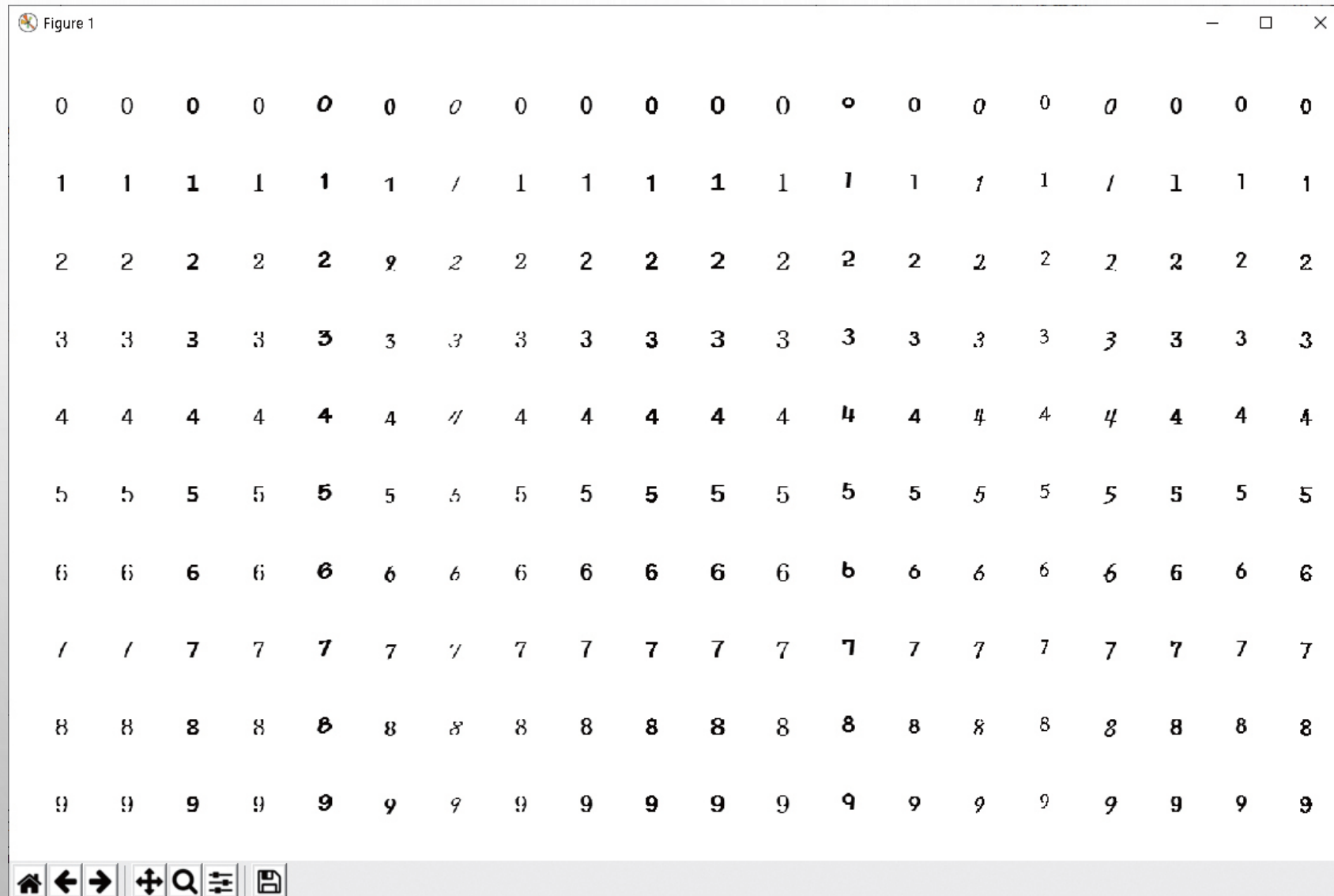
```
01 import cv2
02 import matplotlib.pyplot as plt
03
04 def get_cell(img, j, i, size):
05     x, y = (j * size[0], i * size[1])           # 숫자칸 시작좌표
06     return img[y:y+size[1], x:x+size[0]]
07
08 train_image = cv2.imread("images/train_numbers.png", cv2.IMREAD_COLOR)
09 if train_image is None: raise Exception("영상파일 읽기 에러")
10 train_image = train_image[5:405, 6:806]         # 상하좌우 여백 제거
11
12 size, K = (40, 40), 15                          # 숫자 영상 크기
13 nclass, nsample = 10, 20                        # 그룹수, 그룹당 샘플수
14 cells = [[get_cell(train_image, j, i, size) for j in range(nclass)] for i in range(nsample)]
15 # cells = [np.hsplit(row, nsample) for row in np.vsplit(train_image, nclass)] # numpy 함수 사용
16 # cells = np.reshape(cells, (-1, 40, 40))
17
18 for i, cell in enumerate(cells):
19     plt.subplot(10, 20, i+1), plt.axis('off'), plt.imshow(cell, cmap='gray')
20 plt.tight_layout(), plt.show()
```

숫자 셀 분리 함수

넘파이 함수 사용방법

2) 학습 영상에서 셀 영상 분리

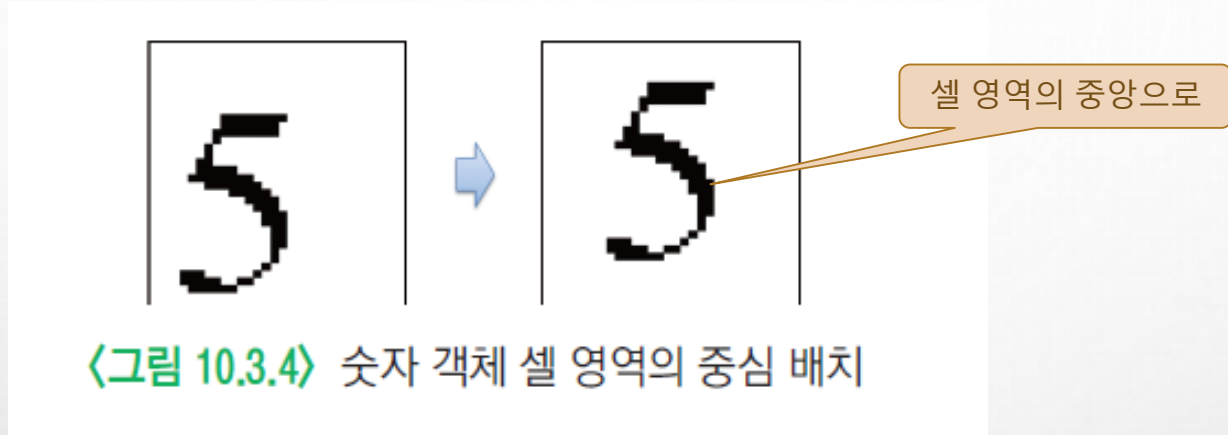
◆ 실행결과 - 분리된 셀 영상 표시



3) 숫자 객체 위치 검색 및 분리

◆ 숫자 객체 중앙 배치

- ❖ 각 셀에서 숫자객체 위치 통일 → 학습 효과와 분류 정확도 향상



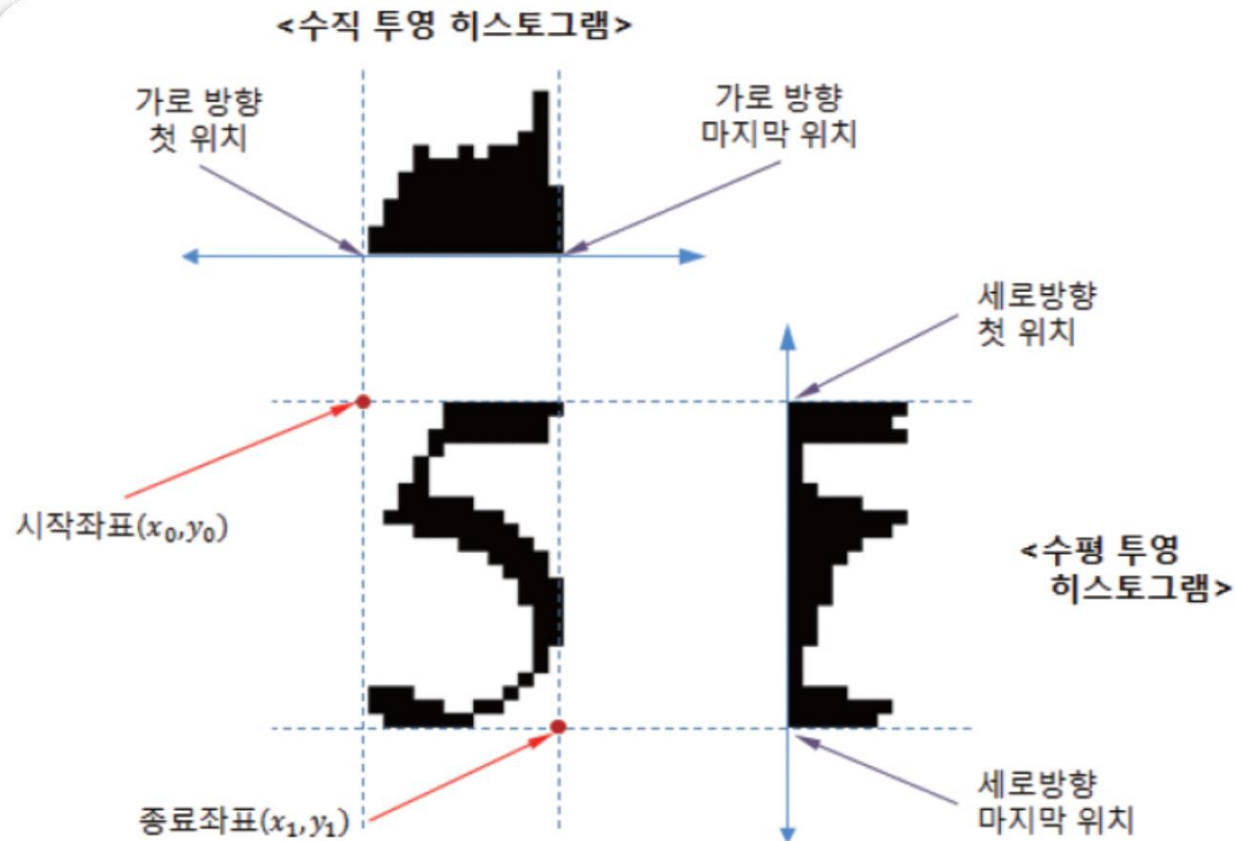
◆ 숫자 객체 위치 인식 – 투영 히스토그램 활용

$$\begin{aligned} histo_v(x) &= \sum_{y=0}^{h-1} f(x, y) \\ histo_v(y) &= \sum_{x=0}^{w-1} f(x, y) \end{aligned} \quad , \quad f(x, y) = \text{좌표}(x, y) \text{의 화소값}$$

3) 숫자 객체 위치 검색 및 분리

◆ 투영 히스토그램을 통한 시작좌표, 종료 좌표 찾기

❖ `cv2.reduce()` 함수 : 가로/ 세로 방향을 감축 → 이 함수로 투영 구현



<그림 10.3.5> 투영을 통한 객체 위치 찾기

3) 숫자 객체 위치 검색 및 분리

Common/knn.py 모듈 일부

```
01 def find_value_position(img, direct):
02     project = cv2.reduce(img, direct, cv2.REDUCE_AVG).ravel() # 투영 히스토그램 계산
03     p0, p1 = -1, -1 # 초기값
04     len = project.shape[0] # 전체 길이
05     for i in range(len):
06         if p0 < 0 and project[i] < 250: p0 = i # 시작 위치 저장
07         if p1 < 0 and project[len-i-1] < 250 : p1 = len-i-1 # 종료 위치 저장
    return p0, p1

10 def find_number(part):
11     x0, x1 = find_histo_position(part, 0) # 수직 투영, x 좌표 찾기
12     y0, y1 = find_histo_position(part, 1) # 수평 투영, y 좌표 찾기
13     return part[y0:y1, x0:x1]
14
```

투영 히스토그램 - 합 대신 평균 이용

초기값

전체 길이

투영 후 1차원 행렬로

투영히스토그램
이용 방법

투영 방향 결정
- 0 : 수직 투영
- 1 : 수평 투영

3) 숫자 객체 위치 검색 및 분리

외곽선 검출 방법

```
15 def find_number2(part): # 외곽선 검출 함수 이용 방법
16     results= cv2.findContours(~part, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
17     contours = results[0] if int(cv2.__version__[0]) >= 4 else results[1]
18     rois = [cv2.boundingRect(contour) for contour in contours] # 외곽선 사각형
19     rois = [(x, y, x+w, y+h) for x, y, w, h in rois] # 사각형을 시작점 종료점으로 변경
20
21     if rois:
22         pts = np.sort(rois, axis=0) # 세로(행) 방향 정렬
23         x0, y0 = pts[0, 0:2] # 분리된 문자 영역 누적
24         x1, y1 = pts[-1, 2:] # 세로 방향 정렬 - 좌표별 정렬
25         w, h = x1-x0, y1-y0 # 시작좌표 중 최소인 x, y 좌표
26         part = part[y0:y0+h, x0:x0+w] # 종료좌표 중 최대인 x, y 좌표
27         return part # 너비, 높이 계산
```

	시작 좌표	종료 좌표
관심 영역1 :	Sx1, Sy1	Ex1, Ey1
관심 영역2 :	Sx2, Sy2	Ex2, Ey2
관심 영역3 :	Sx3, Sy3	Ex3, Ey3
관심 영역4 :	Sx4, Sy4	Ex4, Ey4

세로 방향 오름차순 정렬

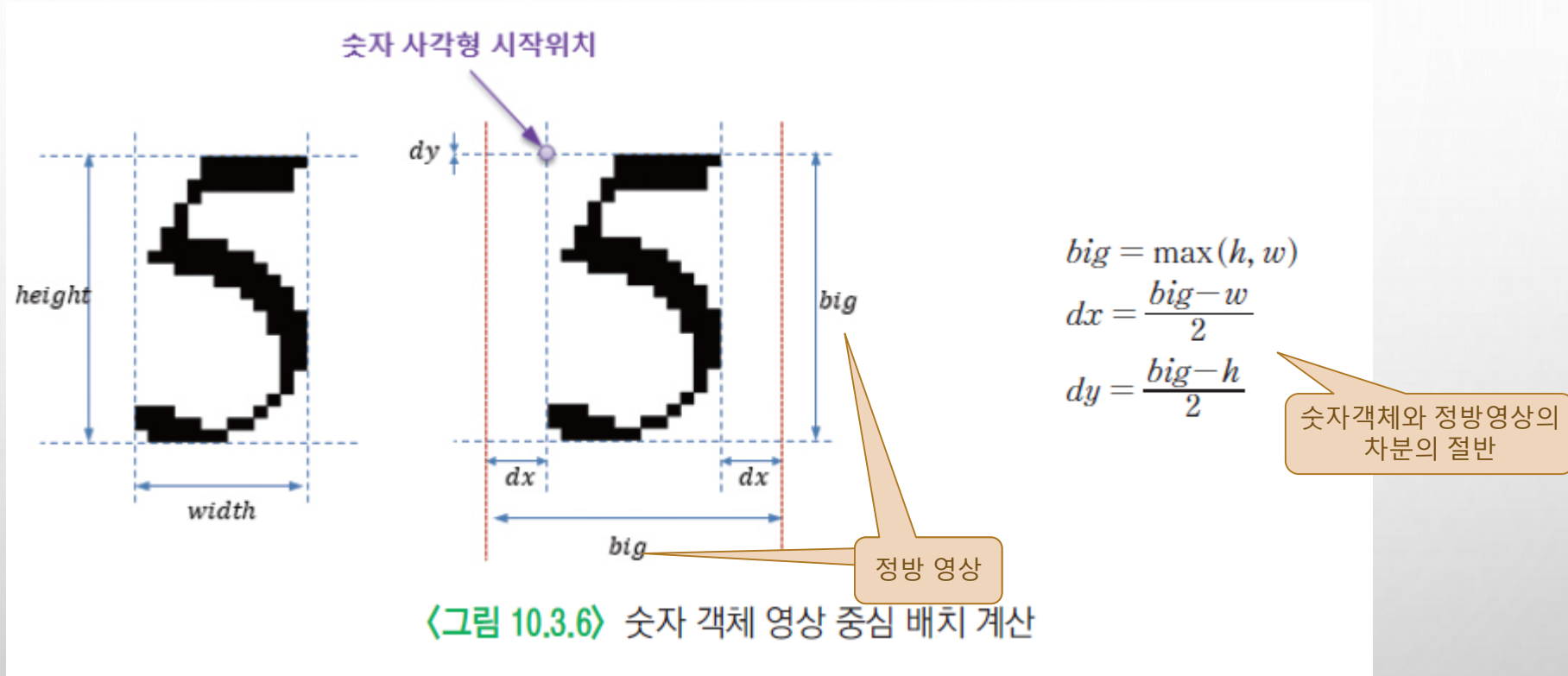
최소 시작 좌표

[0]번 행:	Sx3, Sy2	Ex1, Ey4	← 최솟값들
[1]번 행:	Sx2, Sy1	Ex3, Ey3	
[2]번 행:	Sx1, Sy4	Ex4, Ey2	
[3]번 행:	Sx4, Sy3	Ex3, Ey1	← 최댓값들

최대 종료 좌표

4) 숫자 객체 중앙 배치 및 크기 정규화

◆ 숫자객체를 셀 영역의 중앙에 위치 → `place_middle()` 함수로 구현



4) 숫자 객체 중앙 배치 및 크기 정규화

```
def place_middle(number, new_size):  
    h, w = number.shape[:2]  
    big = max(h, w)  
    square = np.full((big, big), 255, np.float32) # 실수 자료형  
    dx, dy = np.subtract(big, (w, h)) // 2  
    square[dy:dy+h, dx:dx+w] = number  
    return cv2.resize(square, new_size).flatten() # 크기변경 및 벡터변환 후 반환
```

중앙 사각형
시작위치

정방영상 크기 정규화

학습데이터로 만들기 위해
1차원으로 변경후 반환

5) 최종 예제 프로그램

심화예제 10.3.4 k-NN 분류기를 이용한 숫자 인식 - 07.KNN_number.py

```
01 import numpy as np, cv2
02 import matplotlib.pyplot as plt
09 03 from Common.knn import find_number, place_middle          # 관련 함수 импорт
10 04                                                            # 샘플수
11 05 train_image = cv2.imread("images/train_numbers.png", cv2.IMREAD_GRAYSCALE)
12 06 if train_image is None: raise Exception("영상파일 읽기 에러")
13 07 train_image = train_image[5:405, 6:806]                    # 여백 제거
14 08
15 14 nums = [find_number(c) for c in np.reshape(cells, (-1, 40, 40))]
16 15 trainData = np.array([place_middle(n, size) for n in nums])
17 16 labels= np.array([ i for i in range(nclass) for j in range(nsamples) for k in range(1) ])
18 17
19 18 print("cells 형태:", np.array(cells).shape)
20 19 print("nums 형태:", np.array(nums).shape)
21 20 print("trainData 형태:", trainData.shape)
22 21 print("labels 형태:", labels.shape)
23 22
```

셀 영상

숫자 영상

이진화

1행 학습데이터들 구성

5) 최종 예제 프로그램

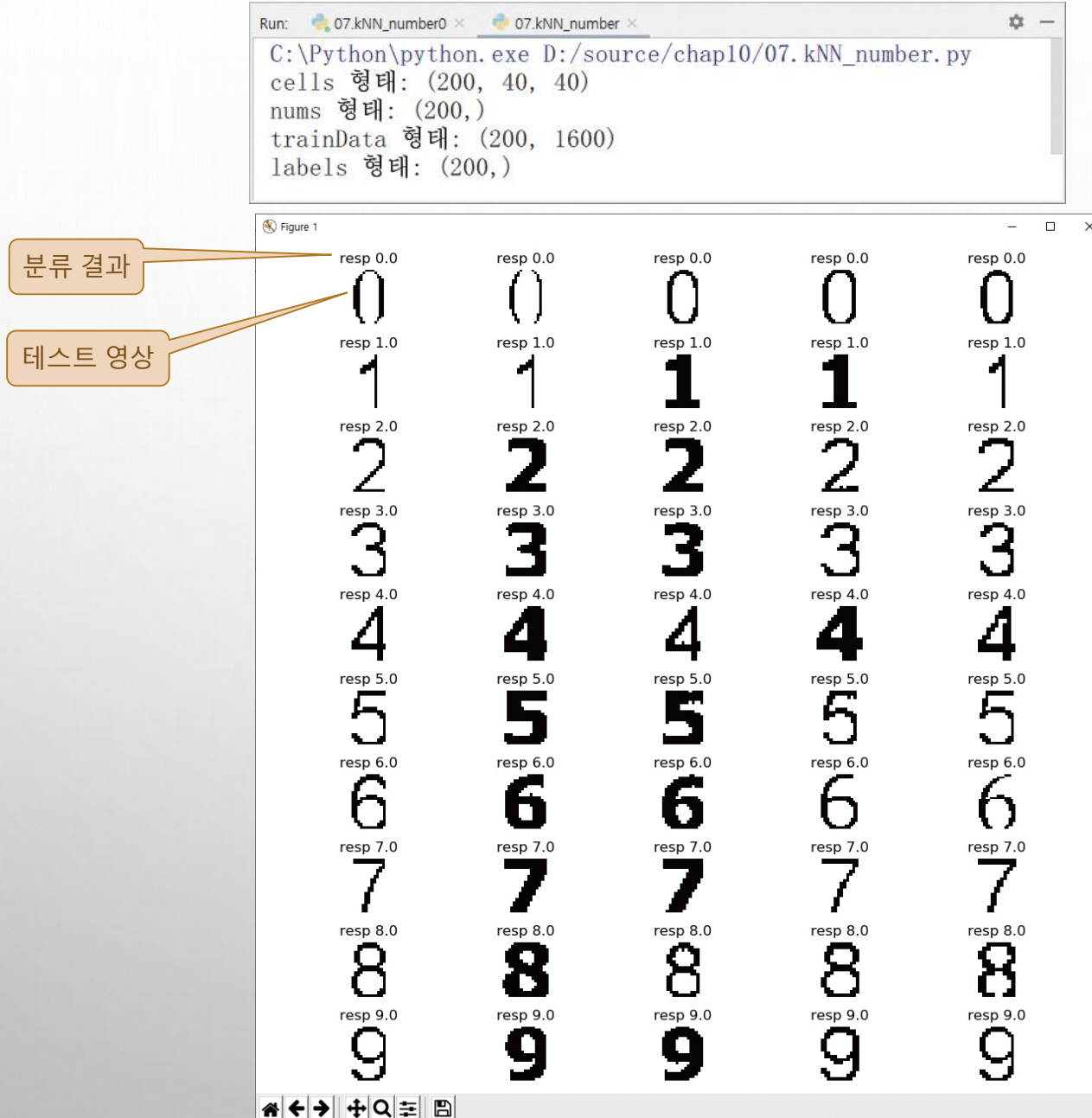
```
23 knn = cv2.ml.KNearest_create()
24 knn.train(trainData, cv2.ml.ROW_SAMPLE, classLabel) # k-NN 학습 수행
25
26 plt.figure(figsize=(10,10))
27 for i in range(50):
28     test_img = cv2.imread("images/num/%d%d.png" % (i / 5, i % 5), 0)
29     cv2.threshold(test_img, 128, 255, cv2.THRESH_BINARY, test_img) # 이진화
30
31     num = find_number(test_img) # 숫자 객체 가져오기
32     data = place_middle(num, size) # 숫자 객체 중심 배치
33     data = data.reshape(1, -1) # 1행 행렬로 변경
34
35     _, [[resp]], _ = knn.findNearest(data, K) # 숫자 분류 수행
36     plt.subplot(10, 5, i+1), plt.axis('off'), plt.imshow(num, cmap='gray')
37     plt.title("resp " + str(resp))
38 plt.tight_layout(), plt.show()
```

테스트 영상 로드

2차원 행렬로 반환되기에
[]를 씌워 1차원으로 행렬
로 저장

5) 최종 예제 프로그램

◆ 실행결과



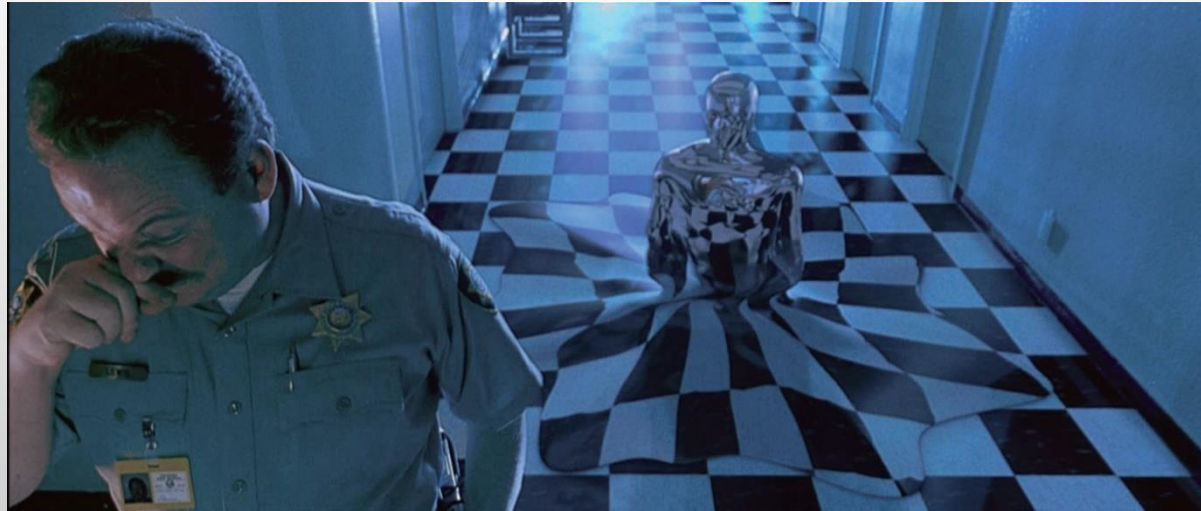
10.4 영상 워핑과 영상 모핑



10.4 영상 워핑과 영상 모핑

◆ 영상 워핑

- ❖ 비선형적인 특정한 규칙에 따라 입력 영상을 재추출하여 영상 형태를 변형하는 기술
- ❖ 나사에서 인공위성으로부터 전송된 일그러진 영상을 복원하는 용도로 처음 사용
- ❖ 일반 크기변경과 달리 크기 변화의 정도가 영상 전체에 대해 균일하지 않음
- ❖ 고무판 위에 영상이 있는 것과 같이 임의의 형태로 구부리는 효과→ 고무 시트 변환
- ❖ 사용 예 - 렌즈 왜곡 보정, 스테레오 영상 정합, 파노라마 영상 합성



10.4 영상 워핑과 영상 모핑

◆ 영상 모핑

- ❖ 하나의 영상에서 형체가 전혀 다른 영상으로 변하도록 하는 기법
- ❖ 두 개의 서로 다른 영상 사이의 변화하는 과정을 서서히 나타내는 것
- ❖ 변형(metamorphosis)이란 단어에서 유래
- ❖ 조지 루카스가 설립한 특수 효과 전문회사인 ILM(Industrial Light and Magic)이 개발



10.4 영상 워핑과 영상 모핑

◆ 예제 - 마우스 드래그로 영상 재배치 하기

$$x' = x + ratio \cdot (pt2.x - pt1.x), \quad ratio = \begin{cases} x < pt1.x & \frac{x}{pt1.x} \\ otherwise & \frac{width - x}{width - pt1.x} \end{cases}$$
$$y' = y$$

10.4 영상 워핑과 영상 모핑

예제 10.4.1 마우스 드래그에 반응하는 워핑 변환 - 08.warping.py

```
01 import numpy as np, cv2
02
03 def morphing():                                # 드래그 거리만큼 영상 왜곡
04     h, w = image.shape[:2]
05     dst = np.zeros((h, w), image.dtype)        # 변환 영상
06     ys = np.arange(0, image.shape[0], 1)       # y 좌표 인덱스
07     xs = np.arange(0, image.shape[1], 0.1)     # x 좌표 인덱스(0.1 간격)
08
09     pt1 = pt2 = None
10     while True:
11         pt1 = pt2 = None
12         while True:
13             pt1 = pt2 = None
14             pt1 = cv2.getMousePos(image)
15             if pt1:
16                 pt2 = cv2.getMousePos(image)
17                 if pt2:
18                     break
19             break
20
21         x1, x10 = pt1[0], pt1[0]*10              # 0.1 간격의 10배→정수 인덱스 구성
22         ratios = xs / x1                          # 기본 변경 비율
23         ratios[x10:] = (w - xs[x10:]) / (w-x1)    # x 좌표 이상은 다른 비율 적용
24
25         dxs = xs + ratios * (pt2[0] - pt1[0])     # 변경 좌표 인덱스 생성
26         xs, dxs = xs.astype(int), dxs.astype(int) # 좌표를 정수값으로 변경
27
28         ym, xm = np.meshgrid(ys, xs)             # 원본 좌표 정방행렬
29         _, dxm = np.meshgrid(ys, dxs)           # 변경 좌표 정방행렬
30         dst[ym, dxm] = image[ym, xm]             # 원본 좌표를 목적 좌표로 매칭
31         cv2.imshow("image", dst)
```

x좌표에
곱해질 비율 계산

원본 x좌표

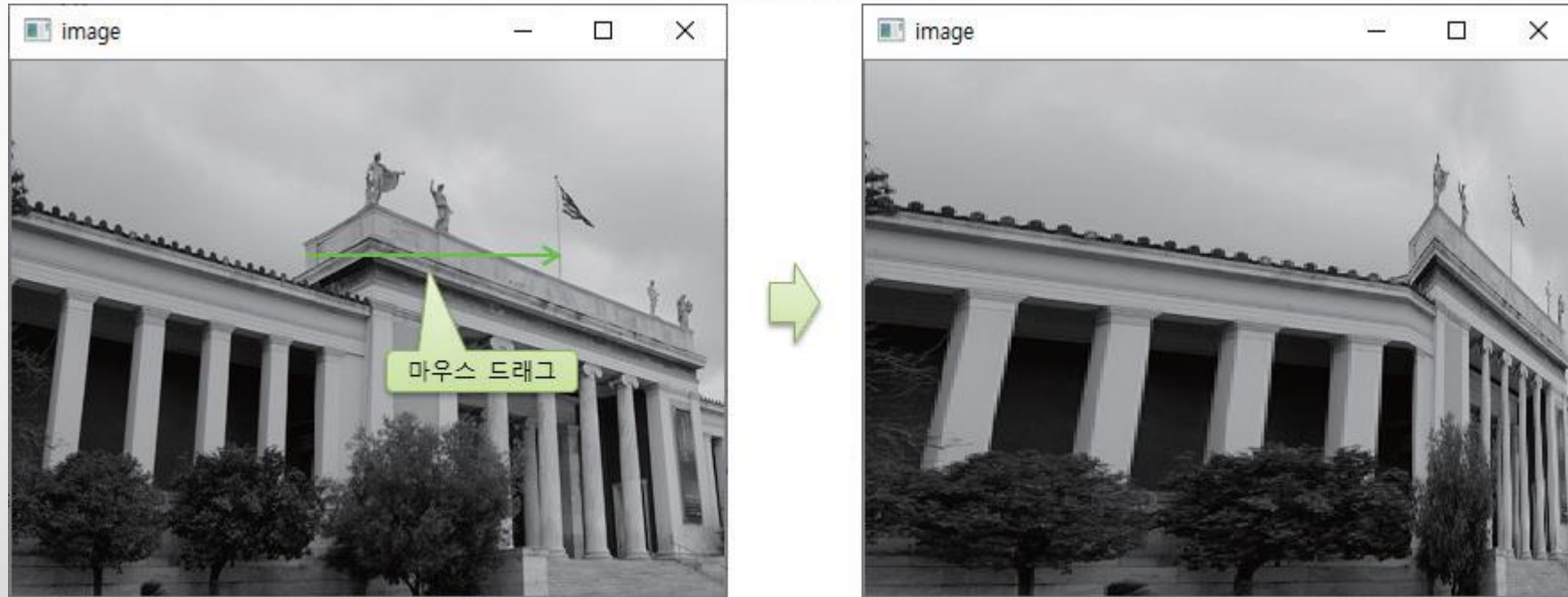
변경 x좌표

10.4 영상 워핑과 영상 모핑

```
21 def onMouse(event, x, y, flags, param):           # 마우스 이벤트 콜백 함수
22     global pt1, pt2
23     if event == cv2.EVENT_LBUTTONDOWN:           # 마우스 왼쪽버튼 누르기 시 시작좌표 지정
24         pt1 = (x, y)                             # 드래그 시작 좌표
25     elif event == cv2.EVENT_LBUTTONUP:           # 마우스 왼쪽버튼 떼기 시 종료좌표 지정
26         pt2 = (x, y)                             # 드래그 종료 좌표
27         morphing()                               # 드래그 종료 시 워핑 변환 수행
28     elif event == cv2.EVENT_RBUTTONDOWNCLK:
29         pt1 = pt2 = (-1, -1)
30         cv2.imshow("image", image)               # 오른쪽 버튼 더블 클릭 시 원복
31
32 image = cv2.imread("images/warp.jpg", cv2.IMREAD_GRAYSCALE)
33 if image is None: raise Exception("영상파일 읽기 에러")
34
35 pt1 = pt2 = (-1, -1)
36 cv2.imshow("image", image)
37 cv2.setMouseCallback("image", onMouse, 0)         # 마우스 콜백 함수 등록
38 cv2.waitKey(0)
```


10.4 영상 워핑과 영상 모핑

◆ 실행결과



10.4 영상 워핑과 영상 모핑

◆ 카메라로 찍은 영상의 왜곡 요인

❖ 외부 파라미터

- 3차원의 실세계 영상을 2차원의 평면 영상으로 맷히기에 기하학적인 왜곡
- 예) 원근 투시 왜곡

❖ 내부 파라미터

- 카메라 내부의 기구적인 부분에 의한 왜곡
- 렌즈, 초점거리, 렌즈와 이미지 센서가 이루는 각

◆ 카메라 캘리브레이션(camera calibration)

❖ 내부 파라미터 값을 구하는 과정

- 영상 좌표로부터 실세계의 3차원 좌표 계산해야하는 경우
- 실세계의 3차원 좌표를 평면 영상에 투영된 위치로 계산해야하는 경우

❖ 정확한 좌표 계산을 위해 카메라 캘리브레이션 필요

10.4 영상 워핑과 영상 모핑

◆ 핀홀(pinhole) 카메라 모델

$$s \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = A \cdot M \cdot \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & C_x \\ 0 & f_y & C_y \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \end{bmatrix} \cdot \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

2차원 영상 평면 좌표

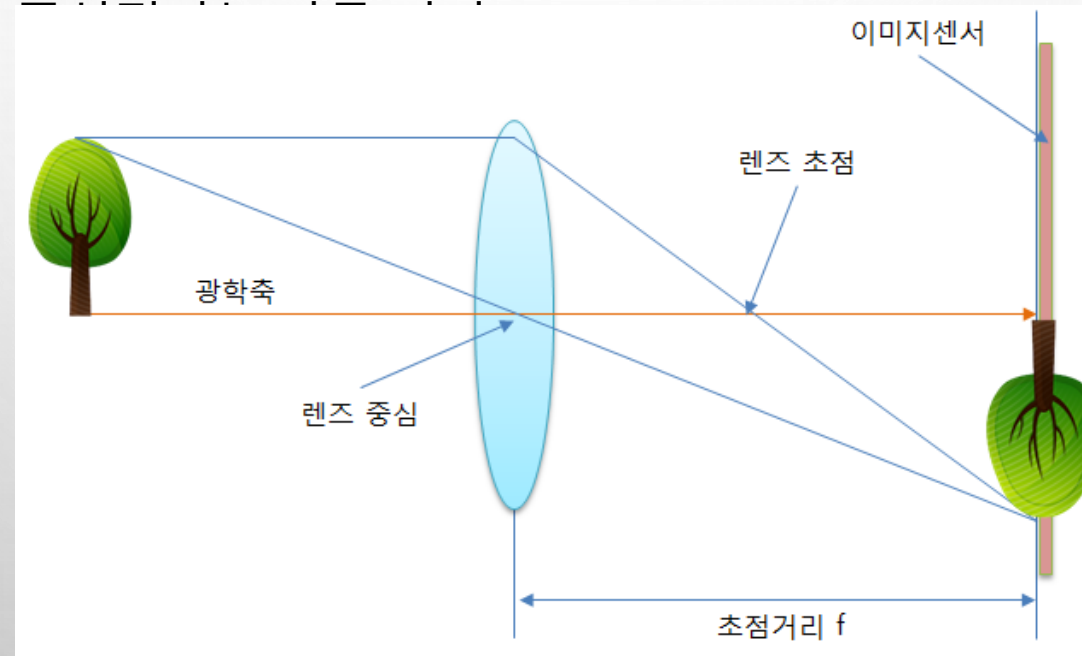
내부 파라미터

외부 파라미터

실세계의 3차원 좌표

10.4 영상 워핑과 영상 모핑

- ❖ 초점거리(f_x, f_y) - 렌즈에서 이미지 센서까지의 거리
 - 디지털 카메라에서는 mm 단위로 표현, 카메라 모델에서는 픽셀(pixel) 단위로 표현
 - 두 개 값으로 표현 - 이미지 센서의 가로, 세로의 셀 간격이 다를 수 있기 때문
- ❖ 주점(C_x, C_y)
 - 카메라 렌즈의 중심에서 이미지 센서에 내린 수선의 영상 좌표
 - 일반적으로 말하는 영상



10.4 영상 워핑과 영상 모핑

예제 10.4.2 카메라 렌즈 왜곡 보정 - 09.warping_camera.py

```
01 import numpy as np, cv2
02 import pickle                                     # 파일 저장/읽기 함수 임포트
03
04 def findCorners(file, bSize):
05     gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
06     ret, corners = cv2.findChessboardCorners(gray, bSize)    # 코너 검출
07     if ret:                                                  # 코너 검출되면
08         criteria = (cv2.TermCriteria_MAX_ITER + cv2.TermCriteria_EPS, 30, 0.1)
09         corners = cv2.cornerSubPix(gray, corners, (11, 11), (-1, -1), criteria) # 부화소로 코너 개선
10     return ret, np.array(corners, np.float32), image
11
12 def show_image(file, bSize, result):
13     cv2.drawChessboardCorners(result[2], bSize, result[1], result[0])    # 코너 표시
14     cv2.imshow(file, result[2])
15
16 def calibrate_correct(objectPoints, imagePoints, image):    # 캘리브레이션 수행 및 보정
17     size = image.shape[1::-1]                                     # 행태와 크기는 역순
18     ret = cv2.calibrateCamera(objectPoints, imagePoints, size, None, None)
19
20     newSacle, roi = cv2.getOptimalNewCameraMatrix(ret[1], ret[2], size, 1) # 재수정
21     undistorted = cv2.undistort(image, ret[1], ret[2], None, newSacle) # 왜곡 보정 수행
22     x, y, w, h = roi
23     return ret, undistorted, undistorted[y:y+h, x:x+w]    # 결과 반환
24
25 bSize = (8, 7)                                                # 체스보드 코너수(가로, 세로)
26 points = [(x, y, 0) for y in range(bSize[1]) for x in range(bSize[0])] # 56행, 3열
27 points = np.array(points, np.float32)                        # 실수형 변경
```

채스판 코너 개수

코너 좌표 개선

코너 좌표
리스트로 구성

10.4 영상 워핑과 영상 모핑

```
29 ## 코너 좌표 및 실세계 3차원 좌표 생성
30 files= ["chessboard_01", "chessboard_02", "chessboard_03"]      # 체스보드 파일명
31 images = [cv2.imread("images/%s.jpg" % file, 1) for file in files]  # 영상 파일들 읽기
32 results = [findCorners(image, bSize) for image in images]        # 검출 결과 리스트
33 imagePoints = [result[1] for result in results if result[0]]      # 코너 좌표 리스트
34 objectPoints = [points] * len(imagePoints)
35
36 [show_image(f, bSize, result) for f, result in zip(files, results) if result[0]]
37
38 image = cv2.imread("images/chessboard_05.jpg", cv2.IMREAD_COLOR)
39 if image is None: raise Exception("영상파일 읽기 에러")
40
41 ret, undistorted, correct_img= calibrate_correct(points, imagePoints, image)
42
43 print("RMS error reported by cv2.calibrateCamera:", ret[0])
44 print("cameraMatrix =\n%s" % ret[1])
45 print("rvecs =\n%s" % np.reshape(ret[3], (3, -1)))
46 print("tvecs =\n%s" % np.reshape(ret[4], (3, -1)))
47
48 with open("camera_calibration.txt", "wb") as f:                    # 결과 저장
49     pickle.dump(ret, f)
50
51 cv2.imshow("original", image)
52 cv2.imshow("undistorted", undistorted)
53 cv2.imshow("cropUndistorted", cropUndistorted)
54 cv2.waitKey(0)
```

실세계 좌표 구성-
points에 []를 씌워서
차원 증가

코너 검출된 영상만 표시

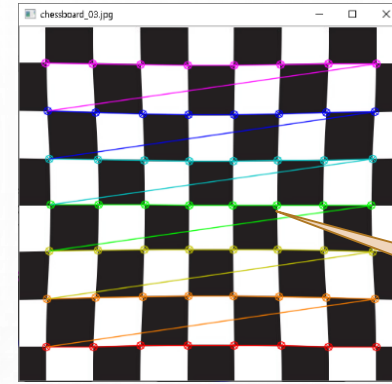
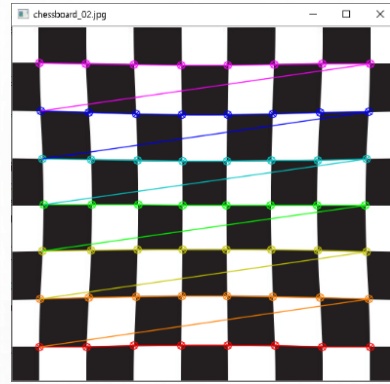
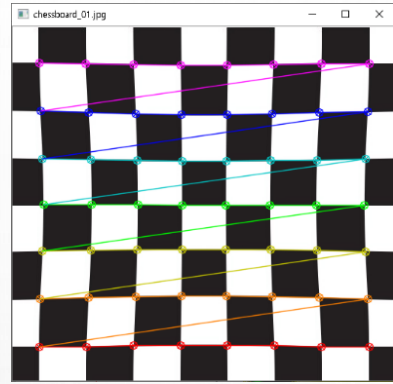
왜곡 보정 함수 호출

3차원 행렬을
2차원 행렬로 변경

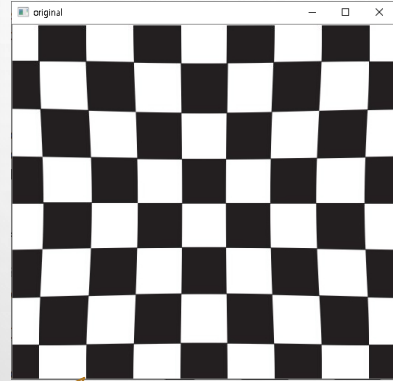
10.4 영상 워핑과 영상 모핑

◆ 실행결과

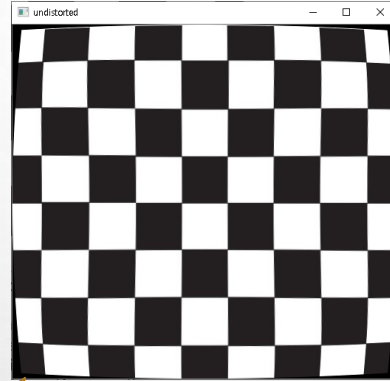
체스모양 입력영상



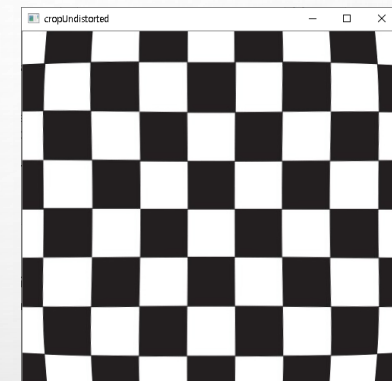
검출된 코너 좌표 표시



보정할 왜곡 영상



보정 영상



외곽 제거 보정영상

```
Run: 10.warping_image x 09.warping x
C:\Python\python.exe D:/source/chap10/10.warping_image.py
RMS error reported by cv2.calibrateCamera: 1.0747286394402353
cameraMatrix =
[[1.24542381e+05 0.00000000e+00 2.30670147e+02]
 [0.00000000e+00 9.53691941e+04 2.40275292e+02]
 [0.00000000e+00 0.00000000e+00 1.00000000e+00]]
distCoeffs =
[[ 1.02053117e+04  8.21750061e-02 -1.91616241e-01 -2.10011508e+00
   6.23675552e-07]]
rvecs =
[[ 1.07996244e+00 -2.61091561e-03 -2.94957182e+00]
 [ 1.07996244e+00 -2.61091561e-03 -2.94957182e+00]
 [ 1.07994551e+00 -2.58412332e-03 -2.94959105e+00]]
rvecs =
[[ 3.03657779  3.02810407 1573.83830054]
 [ 3.03657779  3.02810407 1573.83830054]
 [ 3.03662542  3.02806326 1573.84813004]]
```

심화예제

심화예제 10.4.3

카메라 입력 영상 렌즈 보정 – 10.warping_camera.py

```
01 import numpy as np, cv2
02 from Common.utils import put_string          # 영상에 문자열 표시 함수 импорт
03 from Common.calibration import *             # 앞 예제의 함수 모두 импорт
04
05 bSize, mode = (8, 7), "detect mode"          # 체스보드 코너수, 카메라 모드
06 imagePoints= []                             # 영상 코너 좌표
07 points = [(x, y, 0) for y in range(bSize[1]) for x in range(bSize[0])] # 3차원 좌표들
08 points = np.array(points, np.float32)
09
10 capture = cv2.VideoCapture(0)
11 if capture.isOpened() is False: raise Exception("카메라 연결 안됨")
12 capture.set(cv2.CAP_PROP_BRIGHTNESS, 100)
13 print("카메라 연결 완료")
14
15 while(True):
16     ret, frame = capture.read()               # 카메라 영상 읽기
17     key = cv2.waitKey(30)
18     if key == 27 or ret is False: break       # esc 키: 종료
19     if key == 13: mode = "correct mode"       # 엔터 키: 보정 수행
20     if key == 8: mode = "detect mode"        # 백페이스 키: 코너 검출 수행
21
```

리스트를 ndarray 객체로

심화 예제

```
22     if mode == "detect mode":                                # 코너 검출 모드
23         ret, corners, img = findCorners(frame, bSize)         # 코너 검출
24         if ret: show_image("image", bSize, (ret, corners, img) )    # 코너 표시
25
26         if ret and key == 32:                                    # 스페이스바 키: 검출 코너 저장
27             imagePoints.append(corners)                        # 코너 좌표 저장
28             put_string(frame, "save coord: ", (10, 40), len(imagePoints) ) # 영상에 문자열 표시
29             cv2.imshow("image", frame)
30             cv2.waitKey(500)                                    # 표시 내용 보이기 위해 잠깐 멈춤
31         else:
32             put_string(frame, mode, (10, 40), "")             # 영상에 문자열 표시
33             cv2.imshow("image", frame)
34
35     elif mode == "correct mode":                                # 보정 검출 모드
36         if len(imagePoints) >= 3 :
37             objectPoints = [points] * len(imagePoints)         # 실세계 좌표 n개 구성
38             correct_img = calibrate_correct(objectPoints, imagePoints, frame) # 캘리브레이션
39             put_string(frame, mode, (size[0]-200, 40), "")     # 영상에 문자열 표시
40             cv2.imshow("image", frame)
41         else:                                                    # 검출 코너 부족
42             put_string(frame, "Capture more than 3 corner coordinates", (70, 200))
43             cv2.imshow("image", frame)
44             cv2.waitKey(1000)
45             mode = "detect mode"
46
47     capture.release()
```

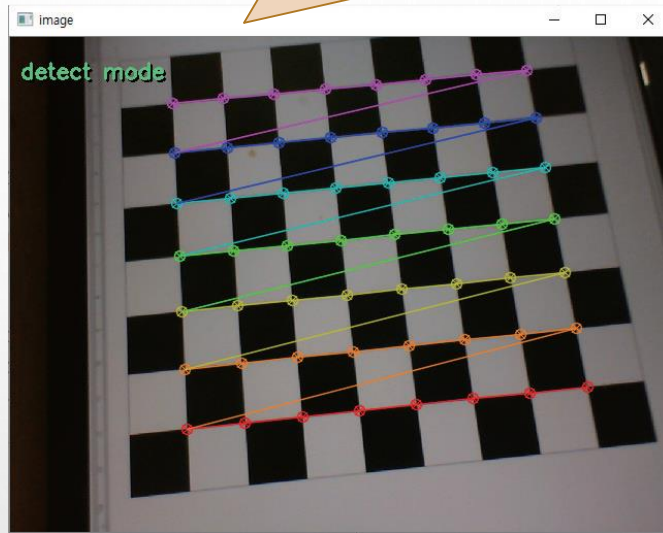
저장 코너 3개 이상
→ 왜곡 보정

에러 메시지

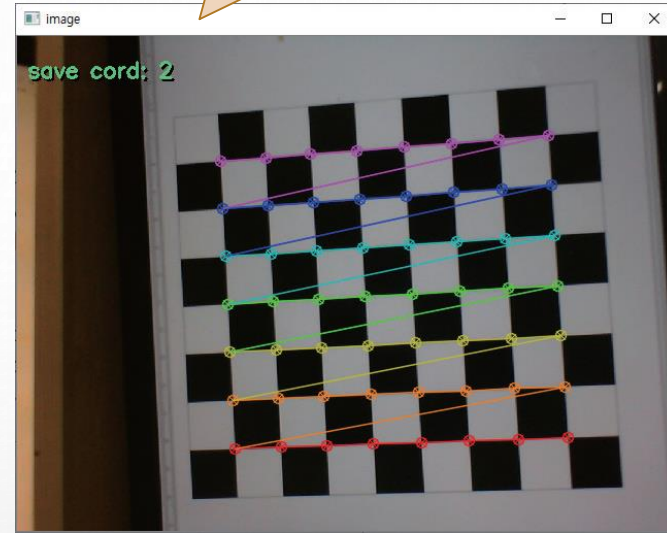
실행결과

◆ 실행 결과

실행후 백스페이스 키 누르면 코너 검출



코너 좌표 저장



백스페이스바

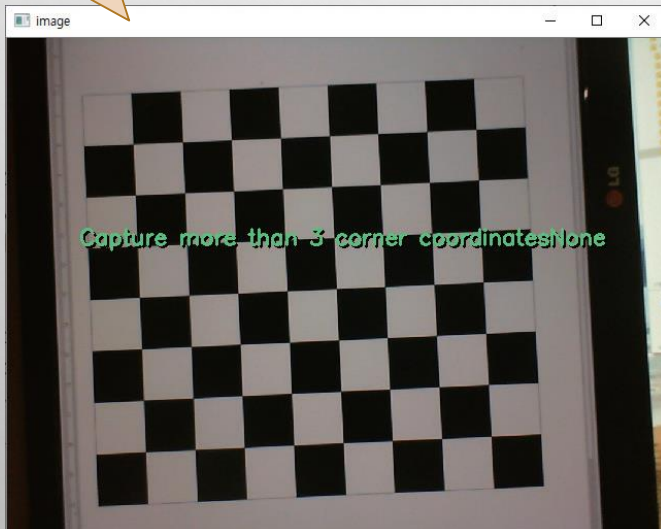


저장 코너 3개 미만

엔터키

엔터키

저장 코너 3개 이상- 왜곡 보정



단원 요약

- ◆ 허프 변환은 영상 내의 선, 원뿐만 아니라 임의의 형태를 지닌 물체를 감지해 내는 대표적인 기술로서 특히 직선 검출에 주로 사용된다. 직교 좌표계로 표현되는 영상의 에지 점들을 극좌표계로 옮겨, 검출하고자 하는 물체의 파라미터(ρ, θ)를 추출하는 방법이다.

$$y = ax + b \leftrightarrow \rho = x \cdot \cos \theta + y \cdot \sin \theta$$

- ◆ 허프 변환은 다음과 같은 세부적인 과정을 거쳐서 수행된다.

1. 극 좌표계에서 누적 행렬 구성
2. 영상 화소의 직선 검사
3. 직선 좌표에 대한 극좌표 누적 행렬 구성
4. 누적 행렬의 지역 최댓값 선정
5. 직선 선별 - 임계값 이상인 누적값 선택 및 내림 차순 정렬

- ◆ 영상 처리에서 중요한 특징 정보로 사용되는 코너는 영상에서 경계가 만나는 지점의 특정한 모양을 갖는 곳을 가리킨다. 이 코너 정보들 중에서 영상의 왜곡에도 불변하는 특징을 가진 지점들이 영상 매칭에 유용하게 사용될 수 있다.
- ◆ 해리스 코너 검출 방법은 영상의 평행이동, 회전 변환에는 불변(invariant)하는 특징이 있고, 어파인(affine) 변환이나 조명(illumination) 변화에도 어느 정도는 강인성이 있다.

단원 요약

- ◆ 최근접 이웃 알고리즘은 기존에 가지고 있는 데이터들을 일정한 규칙에 의해 분류된 상태에서 새로운 데이터의 종류를 예측하는 분류 알고리즘이다. 새로운 미지의 샘플이 입력될때, 학습 클래스의 샘플들과 새 샘플의 거리가 가장 가까운 클래스로 분류한다.
- ◆ 최근접 이웃 방법 중에서 가장 많이 사용되는 것은 학습된 클래스들에서 여러 개의 가까운 이웃을 선출하고 이를 이용하여 미지의 샘플들을 분류하는 방법이다. 이러한 분류 과정을 k-최근접 이웃 분류라고 한다.
- ◆ OpenCV에서는 최근접 이웃 분류를 위해 ml 라이브러리의 KNearest_create() 함수로 kNN 객체(ml_KNearest)를 만들어서 객체 내부 메서드인 train()로 학습을 한다. 또한, findNearest() 메서드로 분류를 수행한다.
- ◆ 영상 워핑은 영상에서 비선형적인 특정한 규칙에 따라 입력 영상을 재추출(resampling)하여 영상의 형태를 변형시키는 기술이다. 영상을 다른 여러 방향으로 늘이거나 크기를조절하는 기법으로 단순한 스케일링과 달리 크기 변화의 정도가 영상 전체에 대해 균일하지 않는 것이 특징이다.
- ◆ 영상 모핑(Morping)은 조지 루카스가 설립한 특수 효과 전문회사인 ILM(Industrial Light and Magic)이 개발한 기법으로 변형(metamorphosis)이란 단어에서 유래되었다. 이것은 하나의 영상에서 형체가 전혀 다른 영상으로 변하도록 하는 기법을 말한다